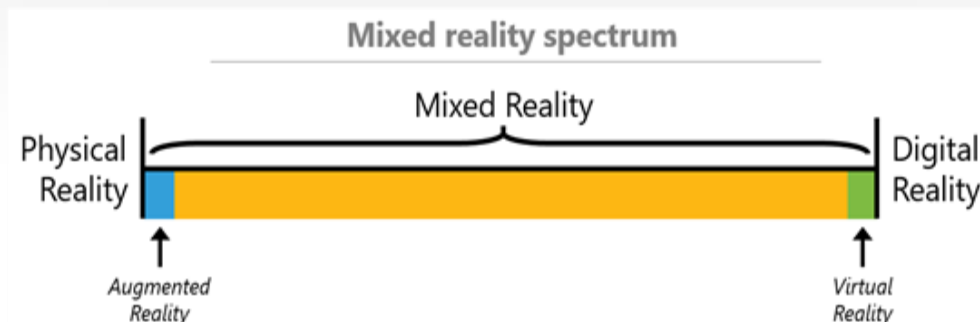




课程预备知识

计算机系 张松海

课程预备知识



- 三维数字世界如何表示？
- 三维数字世界如何转换为人眼可见的二维画面？
- 这上述过程中，计算机是如何实现的？

课程预备知识

- 计算机图形学（真实感图形学）
 - 将三维场景转换为二维图像展示
 - 为此需要
 - 定义三维几何表示
 - 定义材质、纹理
 - 定义光照与着色
 - 这一过程由绘制或渲染（Rendering）+ 光栅化（Rasterization）实现，这一实现流程是渲染管线

目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

04 着色模型

05 渲染管线

06 课程小作业和大作业

目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

04 着色模型

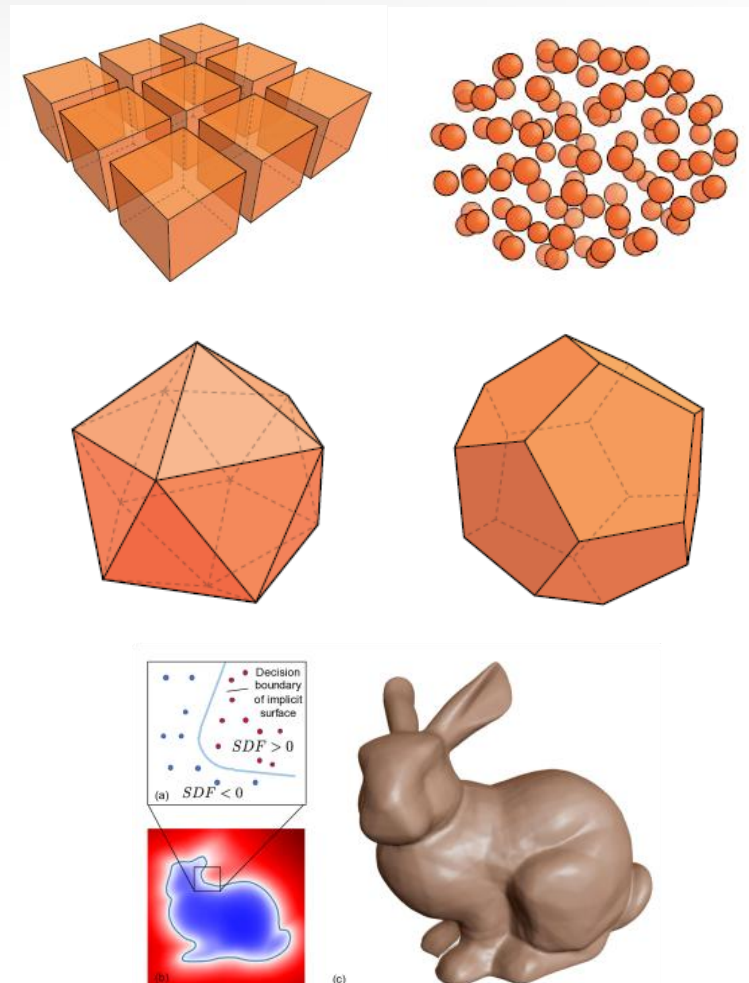
05 渲染管线

06 课程小作业和大作业

三维模型的几何表示

三维模型的几何有多种不同的表示方式：

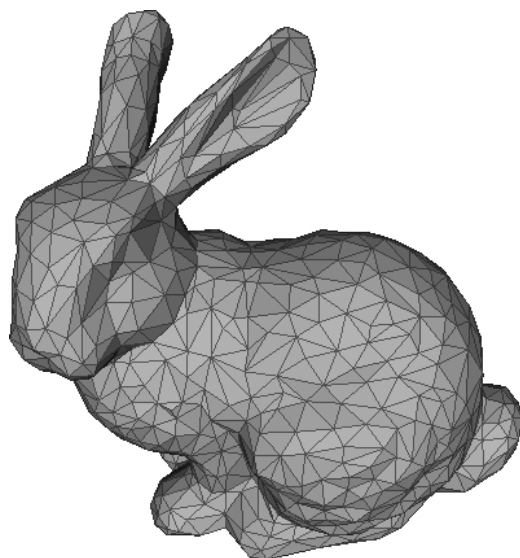
- 显式 (explicit) 表示
 - 体素 (voxel)
 - 点云 (point cloud)
 - 多边形面片 (polygon mesh)
 - ...
- 隐式 (implicit) 表示
 - 距离函数 (distance function)
 - ...



其中，在渲染中使用较多的是**多边形面片**。

三维模型的几何表示

- 多边形面片 (polygon mesh) 是由顶点 (vertices)、边 (edges)、面 (faces) 的集合所定义的几何形状。其中面通常为三角形面 (三角面片, triangle mesh) 或四边形面 (四边形面片, quad mesh)。



三维模型的几何表示

多边形面片常见的存储格式:

- OBJ格式 (Wavefront .obj) : 通常包含顶点坐标 (v)、顶点纹理坐标 (vt)、顶点法向量 (vn)、面 (f) 等信息
- PLY格式 (Polygon File Format)
- STL格式: 只存储几何信息, 没有纹理信息

```
1  # This is a comment
2
3  v 1.000000 -1.000000 -1.000000
4  v 1.000000 -1.000000 1.000000
5  v -1.000000 -1.000000 1.000000
6  v -1.000000 -1.000000 -1.000000
7  v 1.000000 1.000000 -1.000000
8  v 0.999999 1.000000 1.000001
9  v -1.000000 1.000000 1.000000
10 v -1.000000 1.000000 -1.000000
11
12 vt 0.748573 0.750412
13 vt 0.749279 0.501284
14 vt 0.999110 0.501077
15 vt 0.999455 0.750380
16 vt 0.250471 0.500702
17 vt 0.249682 0.749677
18 vt 0.001085 0.750380
19 vt 0.001517 0.499994
20 vt 0.499422 0.500239
21 vt 0.500149 0.750166
22 vt 0.748355 0.998230
23 vt 0.500193 0.998728
24 vt 0.498993 0.250415
25 vt 0.748953 0.250920
26
27 vn 0.000000 0.000000 -1.000000
28 vn -1.000000 -0.000000 -0.000000
29 vn -0.000000 -0.000000 1.000000
30 vn -0.000001 0.000000 1.000000
31 vn 1.000000 -0.000000 0.000000
32 vn 1.000000 0.000000 0.000001
33 vn 0.000000 1.000000 -0.000000
34 vn -0.000000 -1.000000 0.000000
35
36 f 5/1/1 1/2/1 4/3/1
37 f 5/1/1 4/3/1 8/4/1
38 f 3/5/2 7/6/2 8/7/2
39 f 3/5/2 8/7/2 4/8/2
40 f 2/9/3 6/10/3 3/5/3
41 f 6/10/4 7/6/4 3/5/4
42 f 1/2/5 5/1/5 2/9/5
43 f 5/1/6 6/10/6 2/9/6
44 f 5/1/7 8/11/7 6/10/7
45 f 8/11/7 7/12/7 6/10/7
46 f 1/2/8 2/9/8 3/13/8
47 f 1/2/8 3/13/8 4/14/8
```

一个典型的 .obj 文件内容

目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

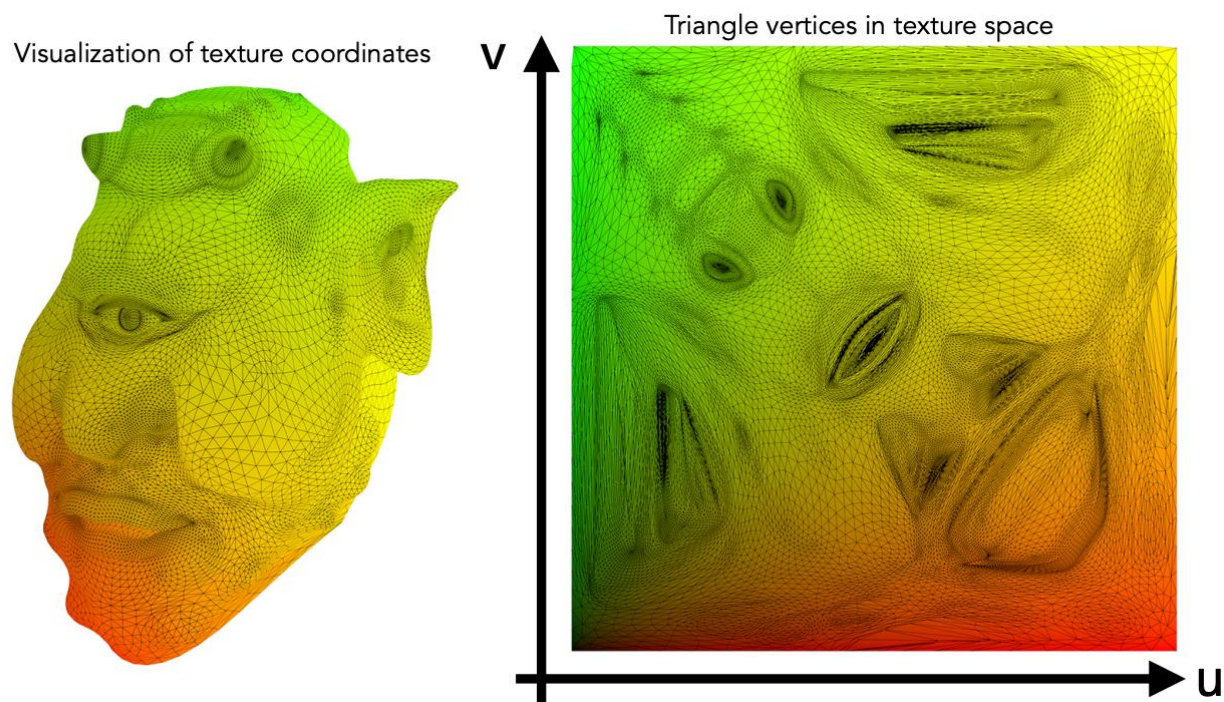
04 着色模型

05 渲染管线

06 课程小作业和大作业

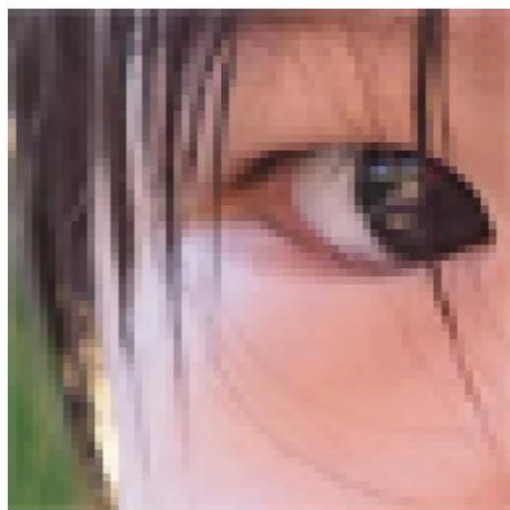
三维模型的纹理表示

- 多边形面片的纹理通常由二维纹理图表示
- 多边形面片的每个顶点对应到二维纹理图上的一个位置，称为该顶点的纹理坐标（或UV坐标），该位置的值为该顶点的纹理值

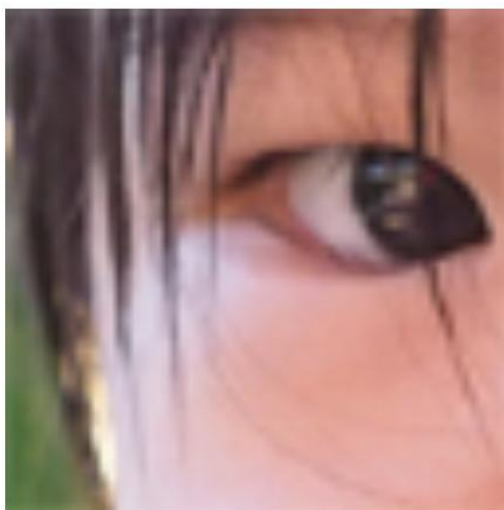


纹理放大 (Texture Magnification)

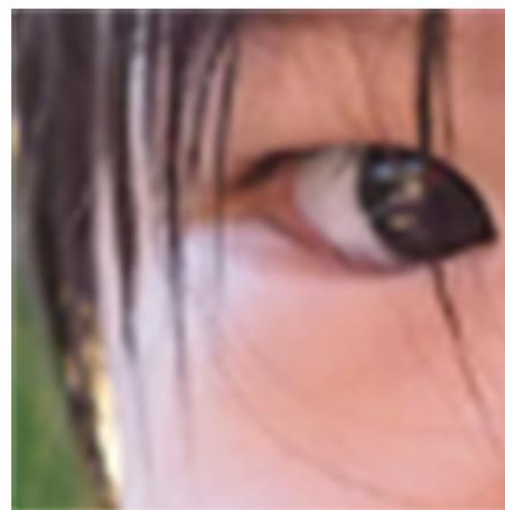
- 纹理图像太小怎么办？插值！
 - 最近邻插值
 - 双线性插值
 - 双三次插值



最近邻插值



双线性插值

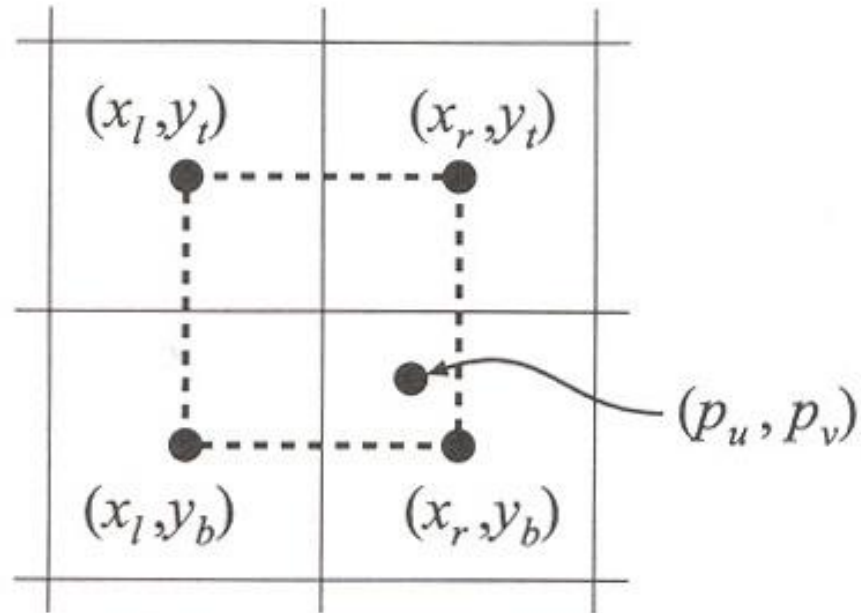


双三次插值

纹理放大 (Texture Magnification)

- 纹理图像太小怎么办？插值！

- 最近邻插值
- **双线性插值**
- 双三次插值

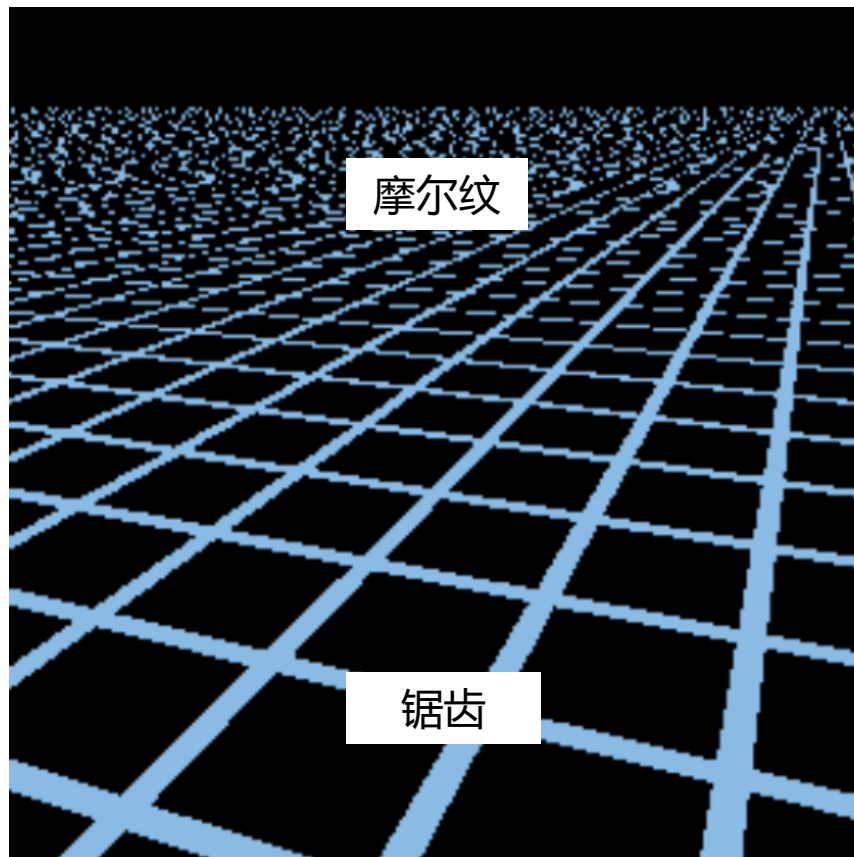
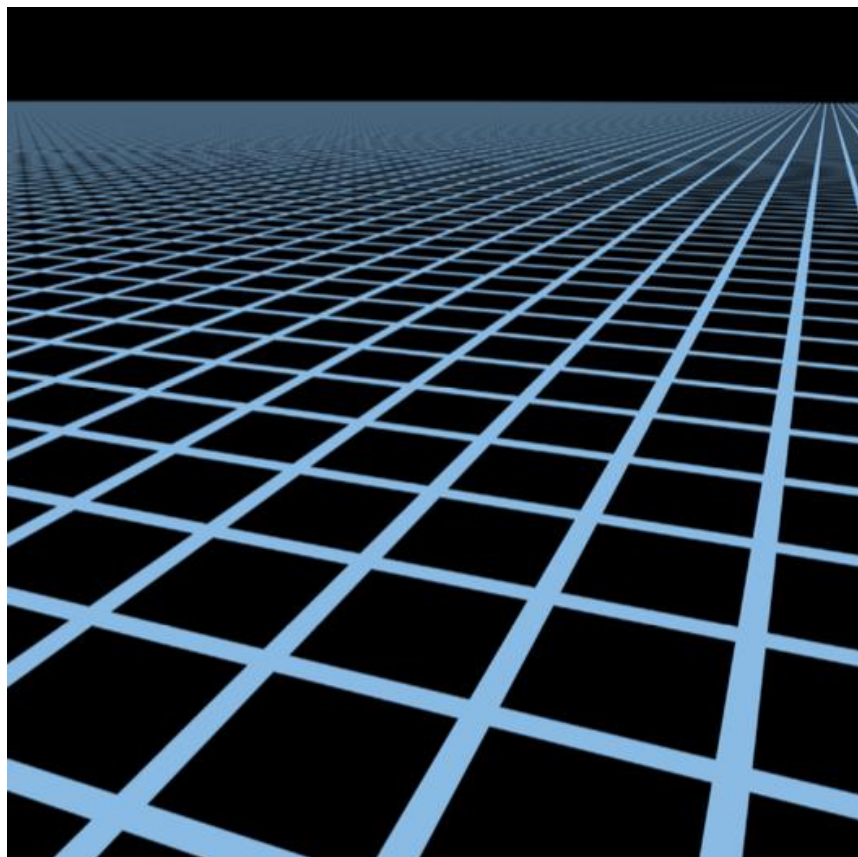


$$\mathbf{b}(p_u, p_v) = (1 - u')(1 - v')\mathbf{t}(x_l, y_b) + u'(1 - v')\mathbf{t}(x_r, y_b) \\ + (1 - u')v'\mathbf{t}(x_l, y_t) + u'v'\mathbf{t}(x_r, y_t)$$

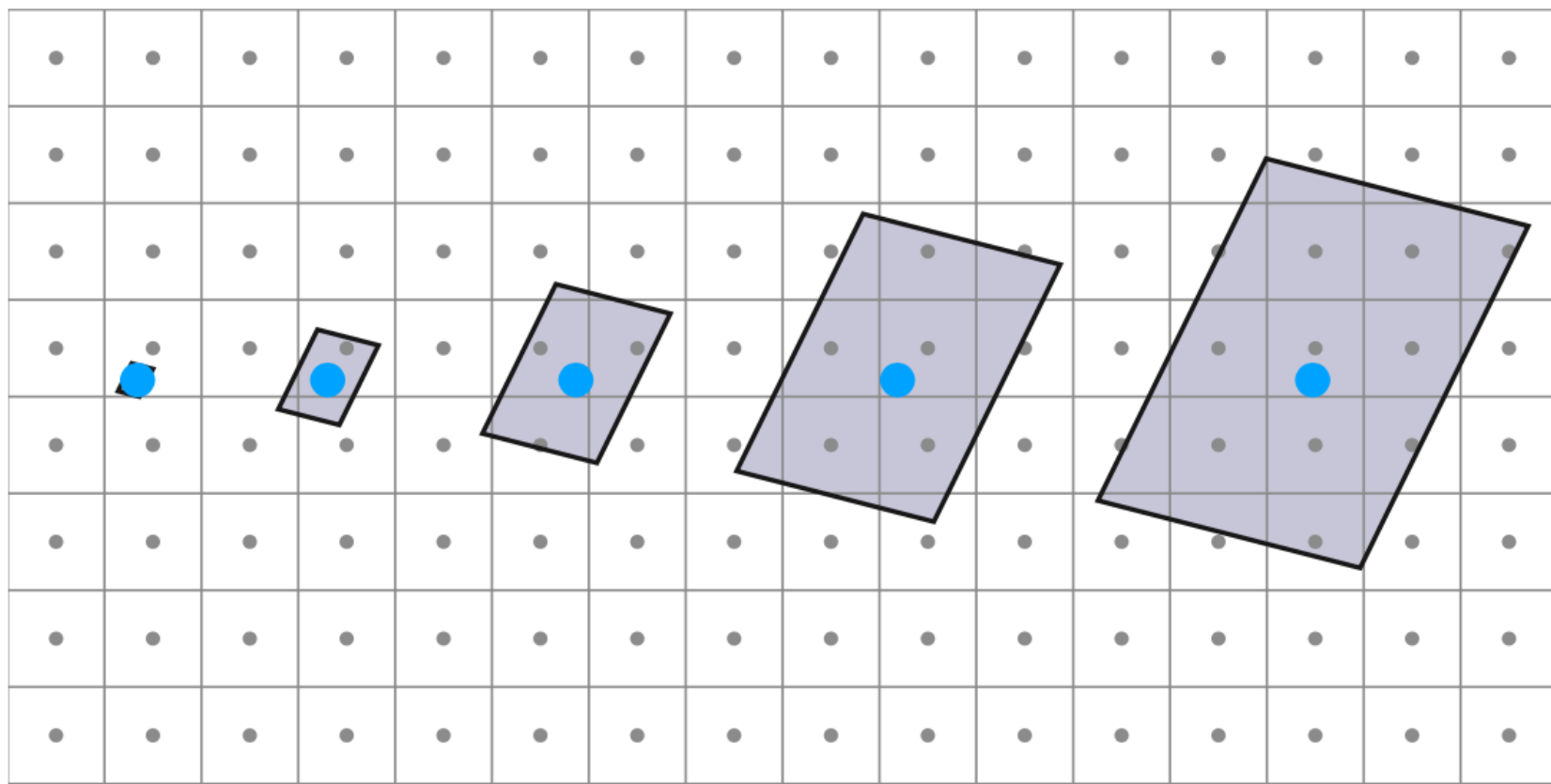
纹理缩小 (Texture Minification)

纹理太大了，怎么办？

如果像纹理放大中一样，单点采样纹理值，则可能会在远处产生摩尔纹（右）



纹理缩小 (Texture Minification)



近
纹理放大
方法：插值

远
纹理缩小
方法：区域查询

纹理缩小 (Texture Minification)

- 为了高效地进行近似范围查询，引入Mipmap——多层次纹理图



Level 0 = 128x128



Level 1 = 64x64



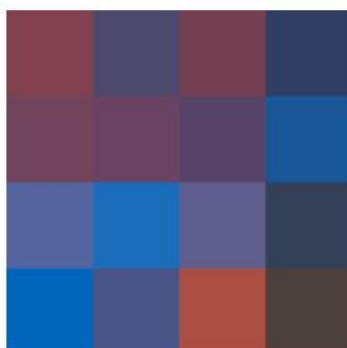
Level 2 = 32x32



Level 3 = 16x16



Level 4 = 8x8



Level 5 = 4x4



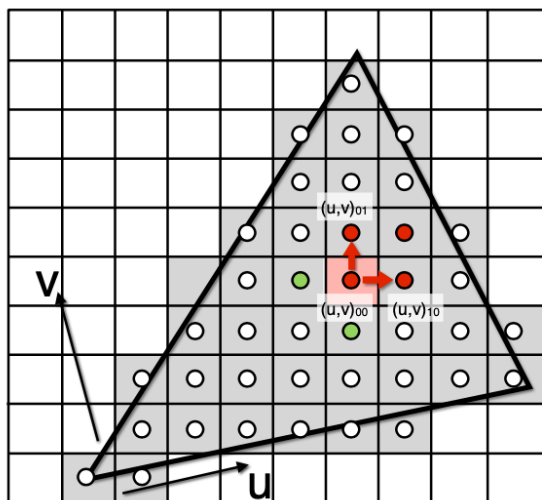
Level 6 = 2x2



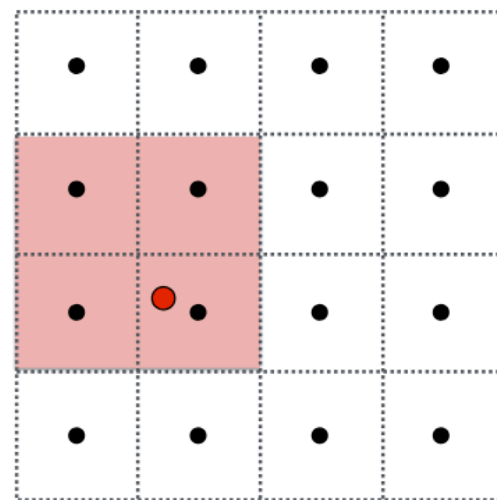
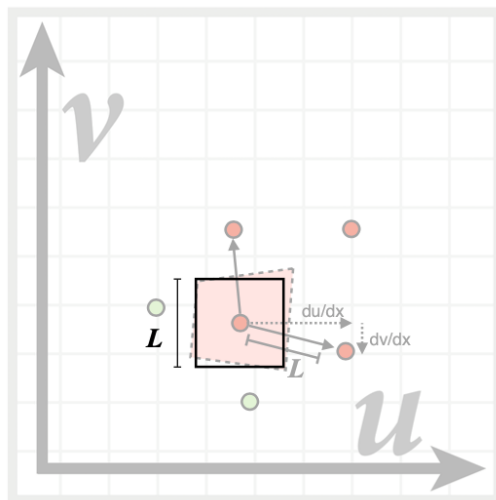
Level 7 = 1x1

Mipmap

- 对于每个像素，估计其对应应在纹理贴图区域的大小，进而对应到Mipmap的层级，在对应层级上进行双线性插值得到纹理值

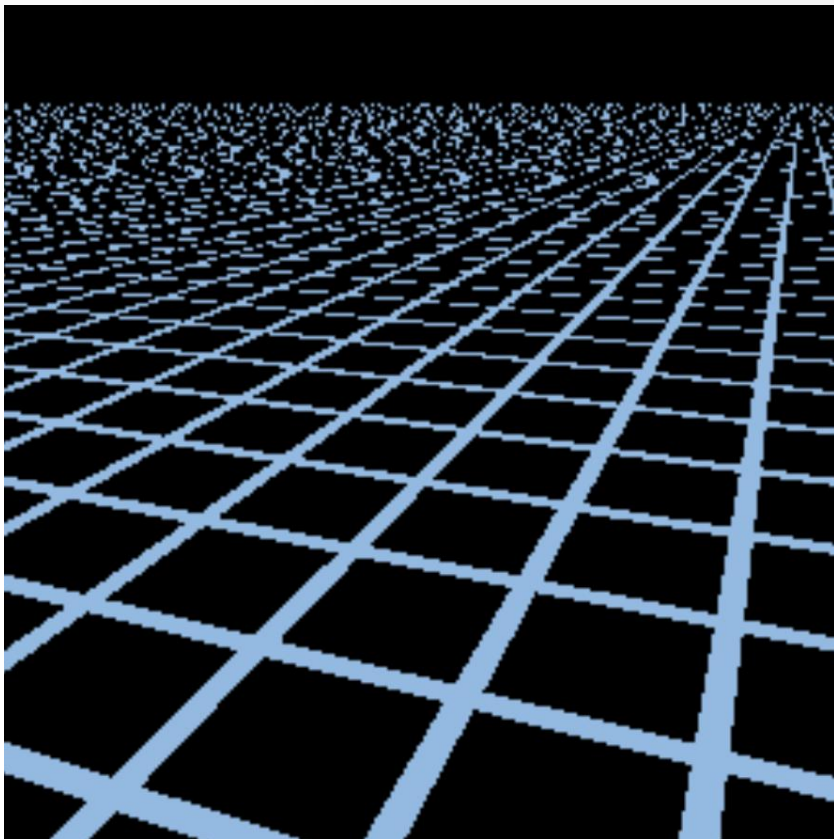


$$D = \log_2 L$$

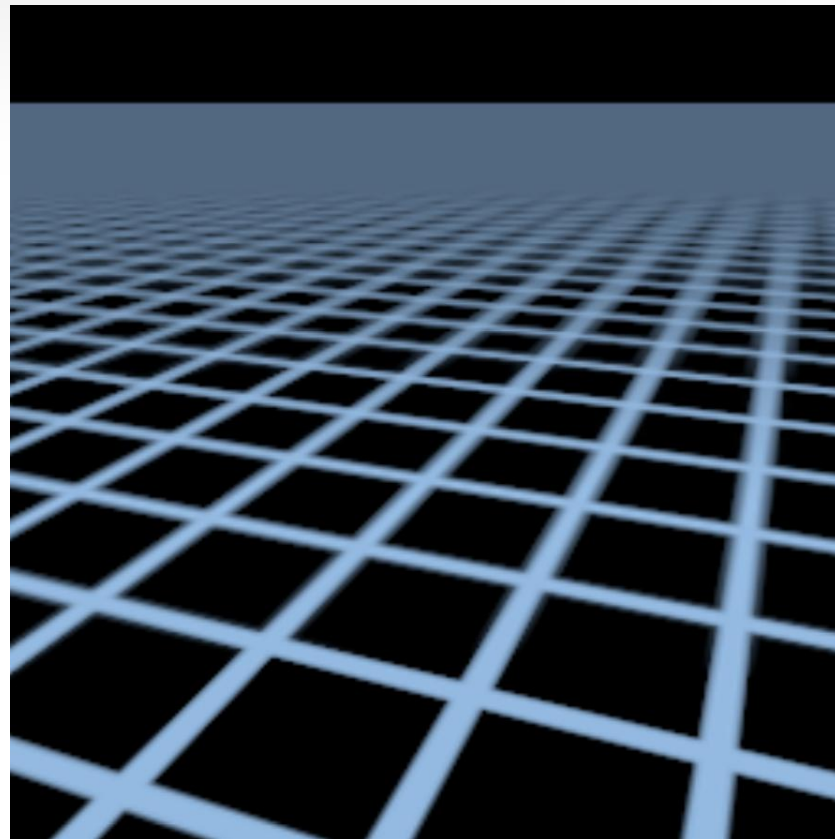


第D层Mipmap纹理图

Mipmap



不使用Mipmap



使用Mipmap

下列哪些属于三维模型的几何属性？

- ☒ A 顶点坐标
- ☐ B 纹理坐标
- ☒ C 顶点法向量
- ☒ D 顶点连接关系
- ☐ E 顶点颜色

提交

目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

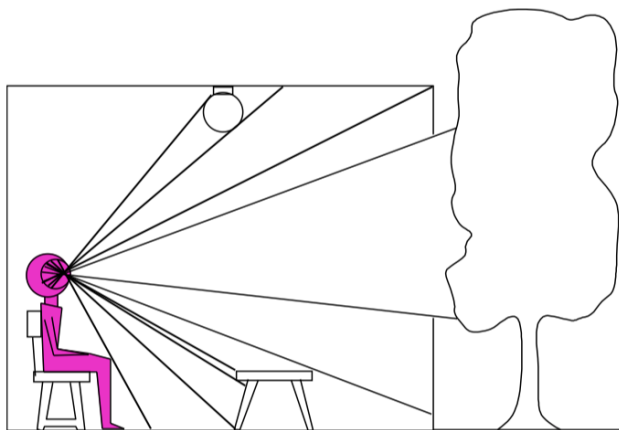
04 着色模型

05 渲染管线

06 课程小作业和大作业

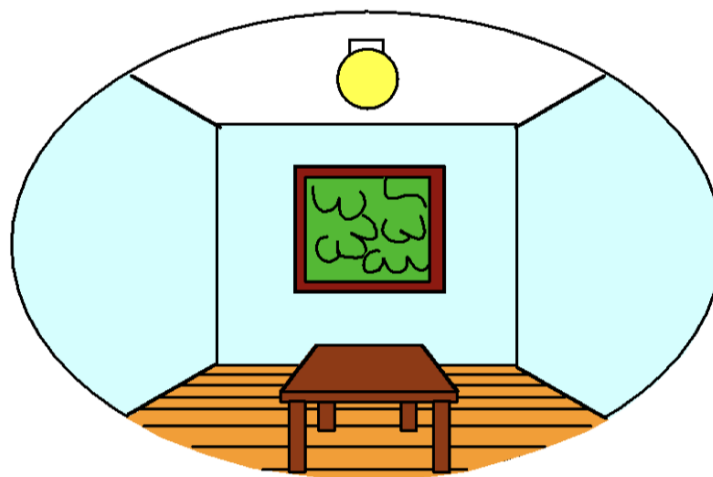
为什么需要变换？

3D world



Point of observation

2D image



Figures © Stephen E. Palmer, 2002

- 图形学关注如何将由几何模型组成的三维场景绘制成高质量的彩色图像
- 变换在图形学中至关重要：通过变换，可以简洁高效地设置和编辑三维场景、改变视点方向

为什么需要变换?

- 假设我们已经有了了一段可以绘制正方形 $[0,1] \times [0,1]$ 的代码:
 - `drawUnitSquare(0,0,1,1);`
- 现在我们需要绘制一个平行于坐标轴, 且左下和右上顶点坐标分别为 (lox, loy) 和 (hix, hiy) 的矩形, 我们应该怎么做?

一种解法

- 一种方法是写一段新的代码：

```
drawRect(lox, loy, hix, hiy) {  
    glBegin(GL_QUADS);  
    glVertex2f(lox, loy);  
    glVertex2f(hix, loy);  
    glVertex2f(hix, hiy);  
    glVertex2f(lox, hiy);  
    glEnd();  
}
```

- 矩形简单可以这么做，可是对于复杂的例子（例如绘制一个茶壶，有大量的面片），怎么办？

利用变换的解法

```
drawRect(lox, loy, hix, hiy) {  
    glTranslate(lox, loy);  
    glScale(hix-lox, hiy-loy);  
    drawUnitSquare(0,0,1,1);  
}
```

- 这样的基于变换的解决方案在图形学中随时都需要用到；使用变换可以使代码更加快速、灵活、模块化

什么叫变换 (Transformation)?

- 变换是一个将空间中的点 x 映射成点 x' 的函数
- 广泛应用于: Morphing, Deformation, Viewing, Projection, Real-time shadows...

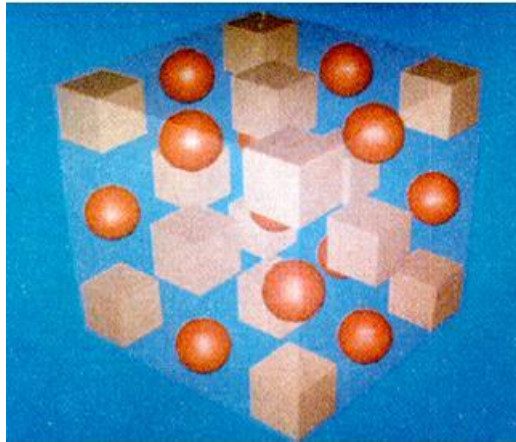


Fig 1. Undeformed Plastic

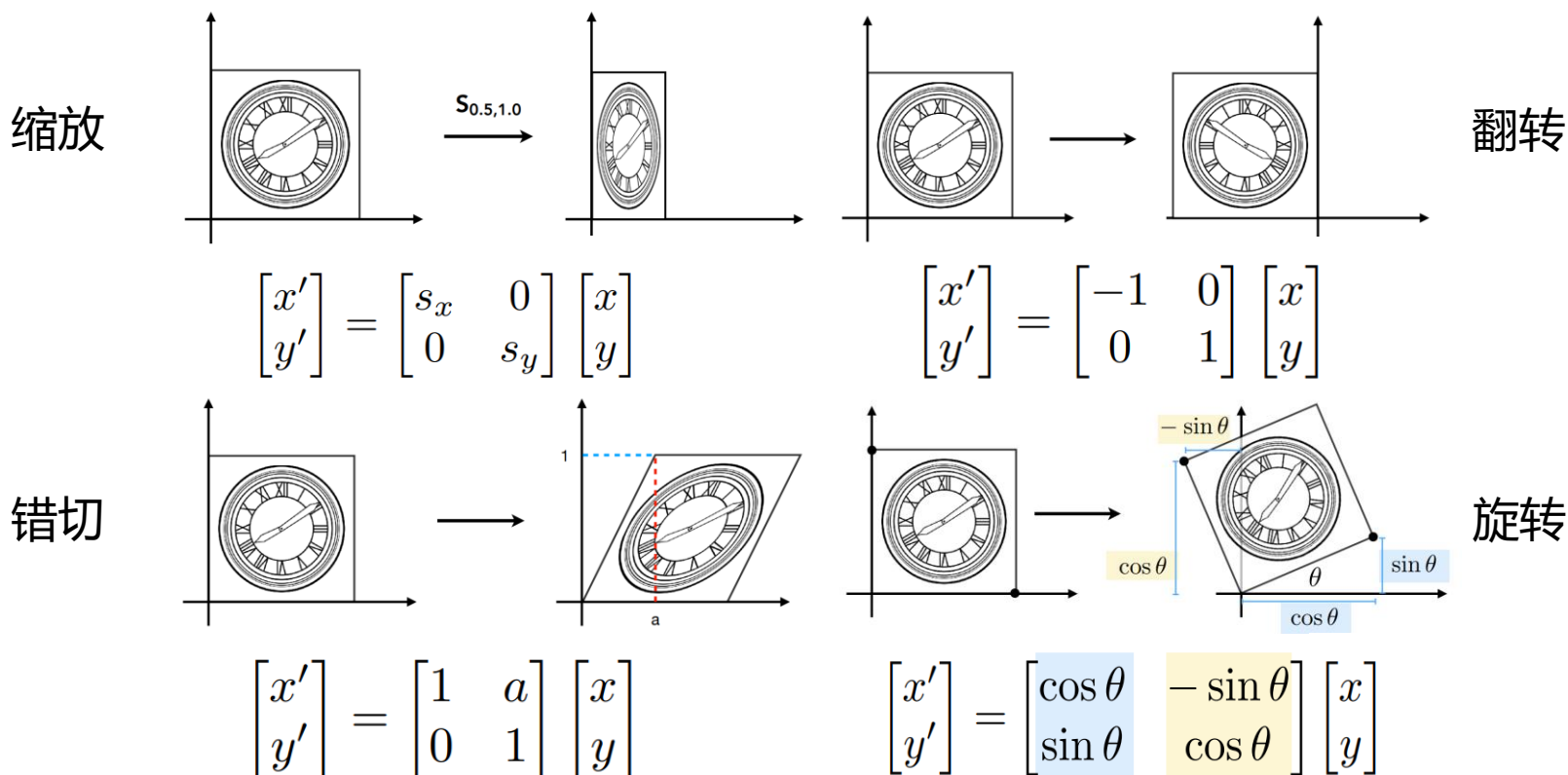


Fig 2. Deformed Plastic

二维空间变换

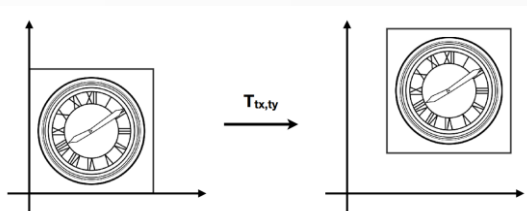
线性变换

- 对于二维空间的点 $x = (x, y)$ ，线性变换可以表示为矩阵乘法的形式 $x' = Mx$ 。缩放、翻转、错切、旋转都是线性变换。



二维空间变换

- 另一个常见的变换：平移，**不是**线性变换



$$\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \end{aligned} \quad \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} + \begin{bmatrix} t_x \\ t_y \end{bmatrix}$$

- 为简化表示和计算，引入齐次坐标 (Homogenous Coordinates)
 - 二维空间点 = $(x, y, 1)^T$
 - 二维空间向量 = $(x, y, 0)^T$
- 在齐次坐标下，平移也可以表示为矩阵乘法形式！**平移和线性变换统称仿射变换 (affine transformation) 。**

$$\begin{pmatrix} x' \\ y' \\ w' \end{pmatrix} = \begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = \begin{pmatrix} x + t_x \\ y + t_y \\ 1 \end{pmatrix}$$

二维空间变换

- 二维空间齐次坐标

- 二维空间点 = $(x, y, 1)$
- 二维空间向量 = $(x, y, 0)^T$

- 性质

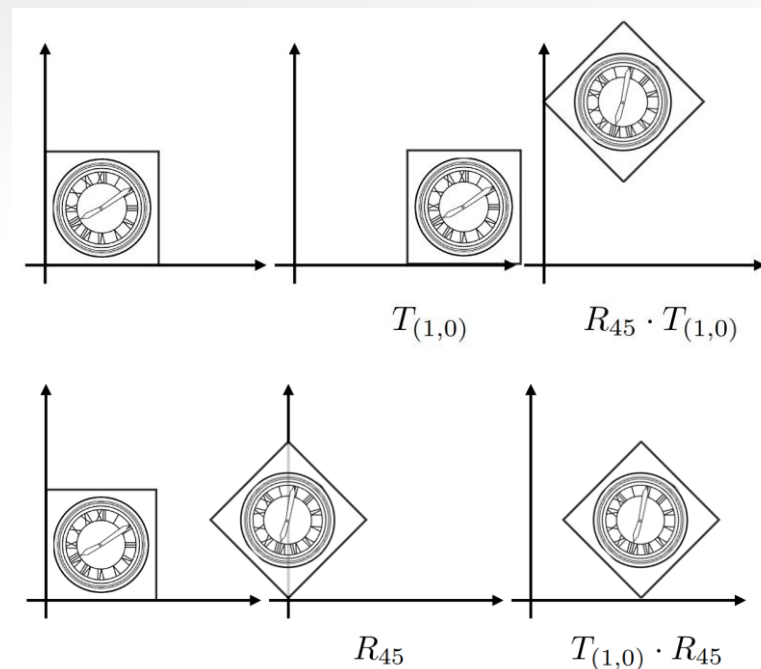
- 向量 + 向量 = 向量
- 点 - 点 = 向量
- 点 + 向量 = 点
- 点 + 点 = ? $(x, y, w)^T = (x/w, y/w, 1)^T, w \neq 0$

二维空间变换

- 由于矩阵乘法不满足交换律，因此变换也不满足交换律，变换顺序至关重要！
- 按 $A_1, A_2, A_3 \dots A_n$ 的顺序应用变换，最后的结果为：

$$A_1 \left(\cdots A_{n-1} \left(A_n(x) \right) \right)$$

$$A_n \left(\cdots A_2 \left(A_1(x) \right) \right)$$



$$R_{45} \cdot T_{(1,0)} \neq T_{(1,0)} \cdot R_{45}$$

三维空间变换

- 三维空间齐次坐标
 - 三维空间点 = $(x, y, z, 1)^T$
 - 三维空间向量 = $(x, y, z, 0)^T$
- $(x, y, z, w)^T = (x/w, y/w, z/w, 1)^T, w \neq 0$

$$\mathbf{S}(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

缩放

$$\mathbf{T}(t_x, t_y, t_z) = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

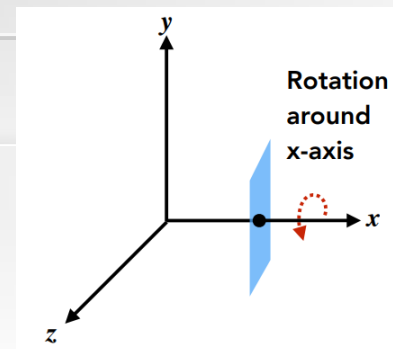
平移

三维空间变换

三维空间的旋转

方向：旋转轴叉乘待旋转向量

$$\mathbf{R}_x(\alpha) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha & 0 \\ 0 & \sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$
$$\mathbf{R}_y(\alpha) = \begin{pmatrix} \cos \alpha & 0 & \sin \alpha & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \alpha & 0 & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$
$$\mathbf{R}_z(\alpha) = \begin{pmatrix} \cos \alpha & -\sin \alpha & 0 & 0 \\ \sin \alpha & \cos \alpha & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



- 绕坐标轴的旋转

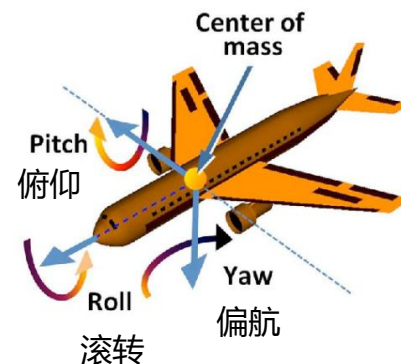
- 任意旋转可表示为绕三个坐标轴旋转的组合（欧拉角）

$$\mathbf{R}_{xyz}(\alpha, \beta, \gamma) = \mathbf{R}_x(\alpha) \mathbf{R}_y(\beta) \mathbf{R}_z(\gamma)$$

- 绕任意轴的旋转

- $\mathbf{v}$ 绕某个轴 $\mathbf{n}$ 旋转角度 α （旋转方向： $\mathbf{n} \times \mathbf{v}$ ）
- 罗德里格旋转公式（Rodrigues' Rotation Formula）

$$\mathbf{R}(\mathbf{n}, \alpha) = \cos(\alpha) \mathbf{I} + (1 - \cos(\alpha)) \mathbf{n} \mathbf{n}^T + \sin(\alpha) \underbrace{\begin{pmatrix} 0 & -n_z & n_y \\ n_z & 0 & -n_x \\ -n_y & n_x & 0 \end{pmatrix}}_{\mathbf{N}}$$



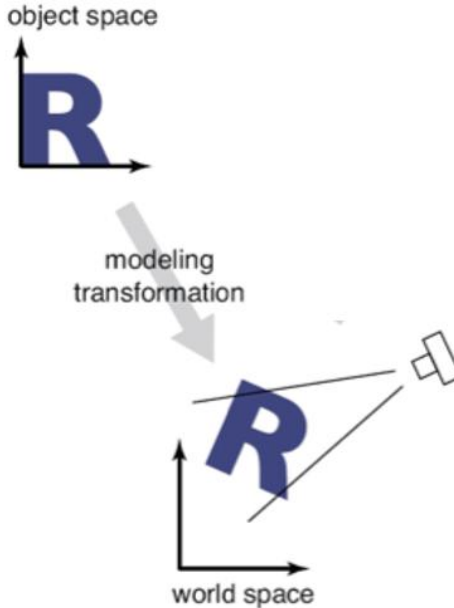
顶点坐标变换

- 讨论单位：三维空间点
- 三维空间点，对应渲染出图像上的哪个像素位置？
- 需要经过模型变换、视角变换、投影变换、视口变换等过程
- 类比拍照的过程
 - 找好拍照的地方，摆好姿势（模型变换）
 - 找好角度（视角变换）
 - 按下快门（投影变换）



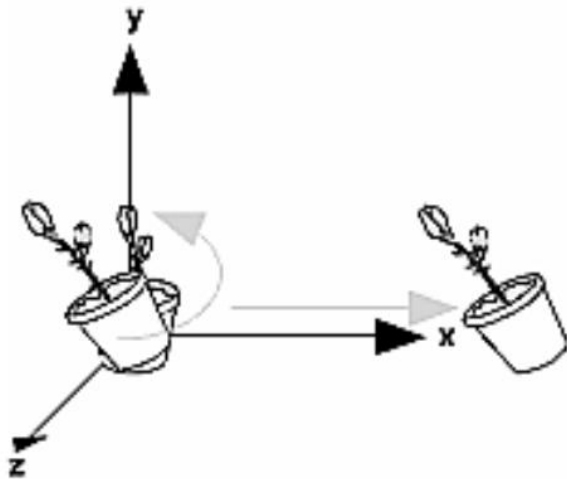
模型变换 (Model Transformation)

- 将模型坐标 (model coordinates) 转换为世界坐标 (world coordinates)
- 即将模型摆在三维空间中的某个位置
- 使用基于齐次坐标的仿射变换 (旋转、平移、缩放) 即可完成

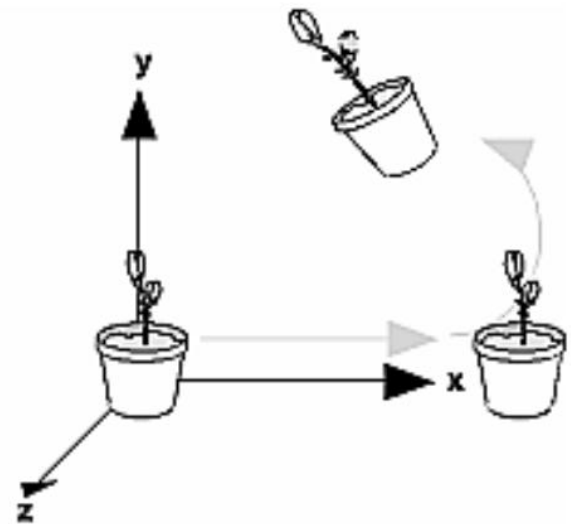


模型变换 (Model Transformation)

- 注意变换顺序



Rotate then Translate



Translate then Rotate

模型变换 – 样例

- 假设有三维空间点 $\mathbf{p} = (2,1,3)^T$ ，依次做如下模型变换：
 - y坐标扩大为3倍，x坐标和z坐标扩大为2倍
 - 绕x轴旋转180度
 - 沿x、y、z轴各平移1个单位
- 在齐次坐标下计算，则以上操作对应的变换矩阵分别为

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 2 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad R = R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos\theta_x & -\sin\theta_x & 0 \\ 0 & \sin\theta_x & \cos\theta_x & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \quad T = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- 模型变换矩阵为 $M = TRS = \begin{pmatrix} 2 & 0 & 0 & 1 \\ 0 & -3 & 0 & 1 \\ 0 & 0 & -2 & 1 \\ 0 & 0 & 0 & 1 \end{pmatrix}$

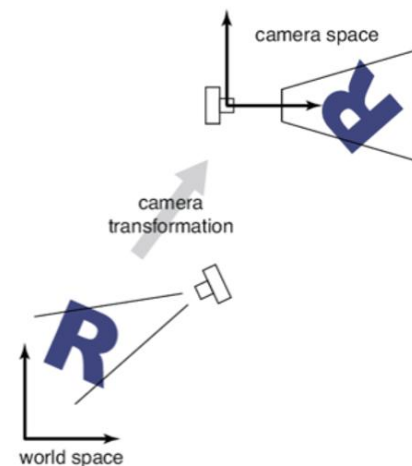
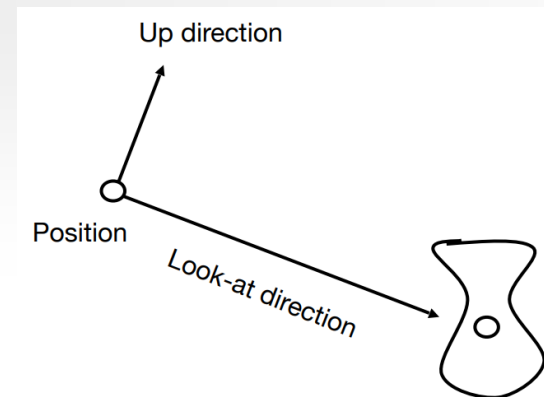
- 模型变换后点的世界空间坐标为 $p_{\text{world}} = Mp = M \begin{pmatrix} 2 \\ 1 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ -5 \\ 1 \end{pmatrix}$

视角变换 (View Transformation)

视角变换 (View Transformation)

- 将世界坐标转换为相机空间坐标
- 描述相机摆放方式
 - 位置 (position) $\mathbf{e}$
 - 朝向 (look-at direction) $\mathbf{g}$
 - 向上方向 (up direction) $\mathbf{t}$
- 为方便起见, 我们固定在相机坐标系中相机位置为原点, 朝向-z轴, 向上方向为y轴
- 则相机坐标系的坐标轴计算如下:

$$- \mathbf{z}^c = \frac{-\mathbf{g}}{\|\mathbf{g}\|}, \quad \mathbf{x}^c = \frac{\mathbf{t} \times \mathbf{z}^c}{\|\mathbf{t} \times \mathbf{z}^c\|}, \quad \mathbf{y}^c = \mathbf{z}^c \times \mathbf{x}^c$$

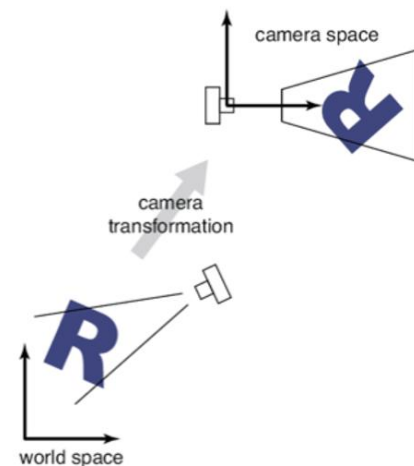
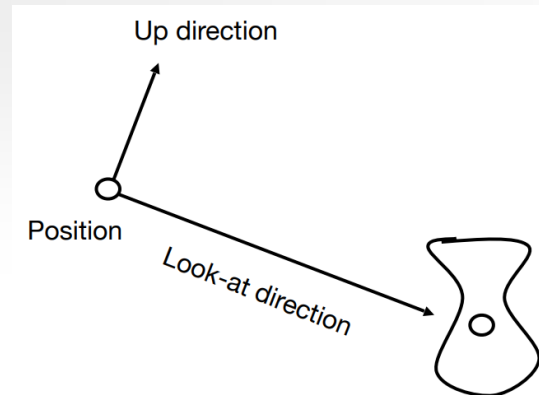


视角变换 (View Transformation)

视角变换 (View Transformation)

- 相机坐标系的坐标轴计算如下：
 - $\mathbf{z}^c = \frac{-\mathbf{g}}{\|\mathbf{g}\|}$, $\mathbf{x}^c = \frac{\mathbf{t} \times \mathbf{z}^c}{\|\mathbf{t} \times \mathbf{z}^c\|}$, $\mathbf{y}^c = \mathbf{z}^c \times \mathbf{x}^c$
- 使用基于齐次坐标的仿射变换即可完成视角变换
- 思考：如何计算变换矩阵？
 - 相机位置平移至原点； $\mathbf{z}^c$ 旋转至 $-\mathbf{z}$ ； $\mathbf{y}^c$ 旋转至 $\mathbf{y}$ ； $\mathbf{x}^c$ 旋转至 $\mathbf{x}$

$$M = R \cdot T(-e) = \begin{pmatrix} x_x^c & x_y^c & x_z^c & -(x_x^c eye_x + x_y^c eye_y + x_z^c eye_z) \\ y_x^c & y_y^c & y_z^c & -(y_x^c eye_x + y_y^c eye_y + y_z^c eye_z) \\ z_x^c & z_y^c & z_z^c & -(z_x^c eye_x + z_y^c eye_y + z_z^c eye_z) \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



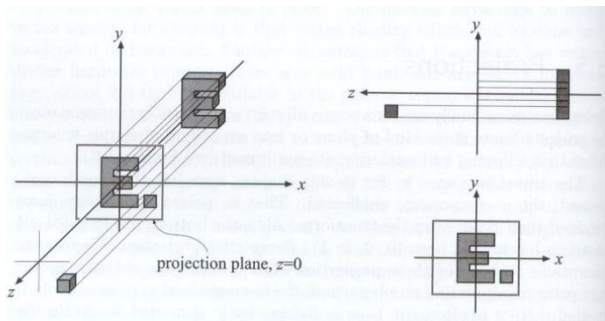
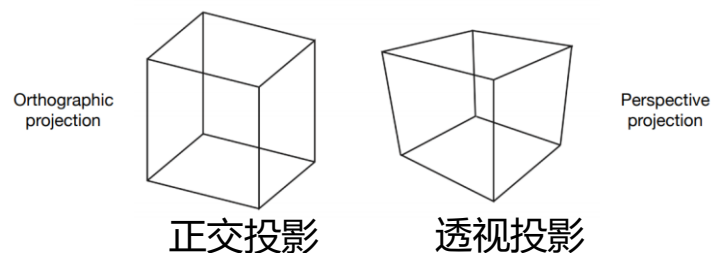
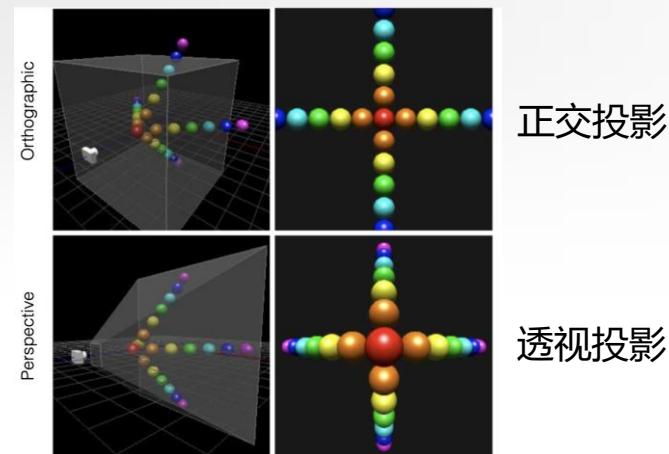
视角变换 – 样例

- 模型变换后点的世界空间坐标为 $p_{\text{world}} = Mp = M \begin{pmatrix} 2 \\ 1 \\ 3 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ -2 \\ -5 \\ 1 \end{pmatrix}$
- 假设有相机参数如下，进行视角变换：相机位置坐标 $\mathbf{e} = (0, 10, 10)$ 相机看向点 $\mathbf{c} = (0, 0, 0)$ 相机上方向 $\mathbf{u} = \left(0, \frac{\sqrt{2}}{2}, -\frac{\sqrt{2}}{2}\right)^T$ 相机看向方向 $\mathbf{g} = \mathbf{c} - \mathbf{e}$
- 计算相机坐标系 $\mathbf{z}^c = \frac{\mathbf{e} - \mathbf{c}}{\|\mathbf{e} - \mathbf{c}\|}$ $\mathbf{x}^c = \frac{\mathbf{u} \times \mathbf{z}^c}{\|\mathbf{u} \times \mathbf{z}^c\|}$ $\mathbf{y}^c = \mathbf{z}^c \times \mathbf{x}^c$
- 则视角变换矩阵 $M_{\text{view}} = R \cdot T(-\mathbf{e}) = \begin{pmatrix} x_x^c & x_y^c & x_z^c & 0 \\ y_x^c & y_y^c & y_z^c & 0 \\ z_x^c & z_y^c & z_z^c & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} 1 & 0 & 0 & -e_x \\ 0 & 1 & 0 & -e_y \\ 0 & 0 & 1 & -e_z \\ 0 & 0 & 0 & 1 \end{pmatrix} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ 0 & \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & -10\sqrt{2} \\ 0 & 0 & 0 & 1 \end{pmatrix}$
- 视角变换后 $p_{\text{view}} = M_{\text{view}} p_{\text{world}} = M_{\text{view}} \begin{pmatrix} 5 \\ -2 \\ -5 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ \frac{3\sqrt{2}}{2} \\ -\frac{27\sqrt{2}}{2} \\ 1 \end{pmatrix}$ $e_{\text{view}} = M_{\text{view}} \mathbf{e} = M_{\text{view}} \begin{pmatrix} 0 \\ 10 \\ 10 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} \rightarrow \text{原点}$
 $g_{\text{view}} = M_{\text{view}} \mathbf{g} = M_{\text{view}} \begin{pmatrix} 0 \\ -10 \\ -10 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ -10\sqrt{2} \\ 0 \end{pmatrix} \rightarrow \text{-z轴}$ $u_{\text{view}} = M_{\text{view}} \mathbf{u} = M_{\text{view}} \begin{pmatrix} 0 \\ \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} \rightarrow \text{y轴}$

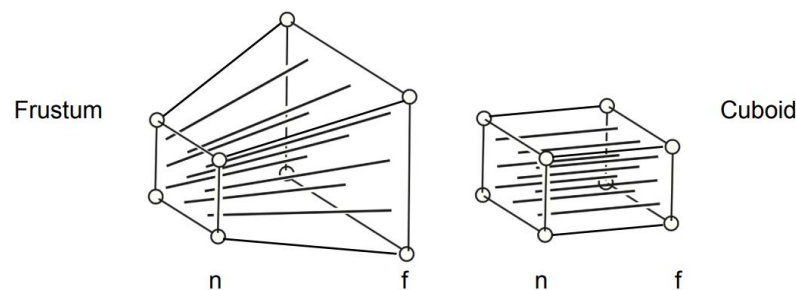
投影变换 (Projection Transformation)

投影变换 (Projection Transformation)

- 三维转二维
- 投影变换将视锥归一化为 $[-1, 1]$ 的立方体, x 、 y 、 z 坐标均在 $[-1, 1]$ 范围内
- 两种投影方式
 - 正交投影: “扔掉” z 坐标即可
 - 透视投影: 先将视锥“挤压”成立方体, 再做正交投影。“挤压”过程虽然不是仿射变换, 但依然可以用矩阵乘法表示 (透视矩阵)。需要额外做一步透视除法。



正交投影过程

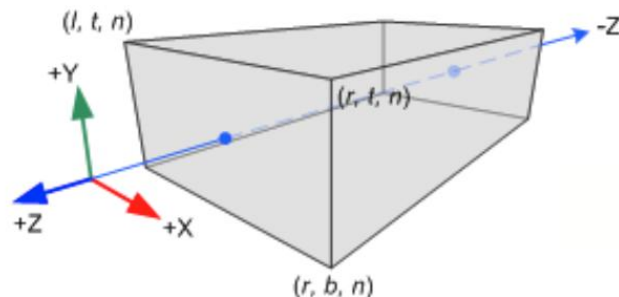


透视投影过程

投影变换 (Projection Transformation)

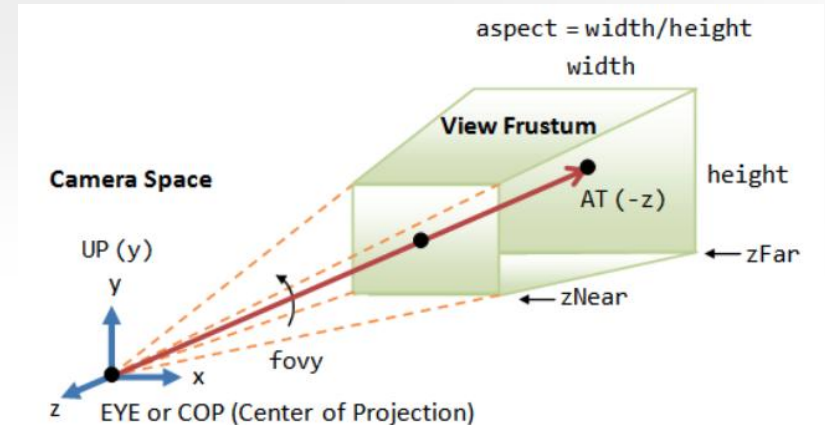
- 正交投影变换
- 参数
 - 近截断平面z坐标 n (near), 远截断平面z坐标 f (far)
 - 近截断平面左、右的x坐标 l (left)、 r (right)
 - 近截断平面上、下的y坐标 t (top)、 b (bottom)
- 变换矩阵

$$M_{proj} = \begin{pmatrix} \frac{2}{r-l} & 0 & 0 & -\frac{r+l}{r-l} \\ 0 & \frac{2}{t-b} & 0 & -\frac{t+b}{t-b} \\ 0 & 0 & \frac{-2}{f-n} & -\frac{f+n}{f-n} \\ 0 & 0 & 0 & 1 \end{pmatrix}$$



投影变换 (Projection Transformation)

- 透视投影变换
- 参数
 - 垂直方向视角fovy
 - 宽高比aspect
 - 近截断平面z坐标 zNear, 远截断平面z坐标 zFar
- 变换矩阵



$$f = \cot(\text{fovy} / 2)$$

$$M_{proj} = \begin{pmatrix} \frac{f}{aspect} & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & -\frac{zFar + zNear}{zFar - zNear} & -\frac{2 \cdot zFar \cdot zNear}{zFar - zNear} \\ 0 & 0 & -1 & 0 \end{pmatrix}$$

透视除法 (Perspective Division)

- 透视变换定义在齐次坐标下 ($w \neq 0$)
- 需要额外做一次透视除法
- 得到的归一化坐标称为标准化设备坐标 (Normalized Device Coordinates, NDC)

$$v_{clip} = \begin{pmatrix} x_{clip} \\ y_{clip} \\ z_{clip} \\ w_{clip} \end{pmatrix} \longrightarrow v_{NDC} = \begin{pmatrix} x_{clip} / w_{clip} \\ y_{clip} / w_{clip} \\ z_{clip} / w_{clip} \\ 1 \end{pmatrix} \begin{matrix} \in (-1,1) \\ \in (-1,1) \\ \in (-1,1) \end{matrix}$$

vertex in clip space

vertex in NDC

透视变换 – 样例

- 视角变换后点的相机空间坐标为 $p_{\text{view}} = M_{\text{view}} p_{\text{world}} = M_{\text{view}} \begin{pmatrix} 5 \\ -2 \\ -5 \\ 1 \end{pmatrix} = \begin{pmatrix} 5 \\ \frac{3\sqrt{2}}{2} \\ -\frac{27\sqrt{2}}{2} \\ 1 \end{pmatrix}$
- 假设有如下参数: $aspect = 1, fovy = 90^\circ, zNear = 2, zFar = 22$. 计算透视投影矩阵、透视变换后点的截断坐标和NDC.

- 透视投影矩阵定义为 $M_{\text{proj}} = \begin{pmatrix} f_x & 0 & 0 & 0 \\ 0 & f_y & 0 & 0 \\ 0 & 0 & \frac{n+f}{n-f} & \frac{2 \cdot f \cdot n}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}$

- 其中 $f_y = \cot\left(\frac{fovy}{2}\right), f_x = f_y / aspect$

- 则

$$M_{\text{proj}} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{6}{5} & -\frac{22}{5} \\ 0 & 0 & -1 & 0 \end{pmatrix} \quad p_{\text{clip}} = M_{\text{proj}} p_{\text{view}} = \begin{pmatrix} 5 \\ \frac{3\sqrt{2}}{2} \\ \frac{81\sqrt{2}-22}{5} \\ \frac{27\sqrt{2}}{2} \end{pmatrix} = \begin{pmatrix} x_{\text{clip}} \\ y_{\text{clip}} \\ z_{\text{clip}} \\ w_{\text{clip}} \end{pmatrix}$$

$$p_{\text{ndc}} = \begin{pmatrix} x_{\text{clip}}/w_{\text{clip}} \\ y_{\text{clip}}/w_{\text{clip}} \\ z_{\text{clip}}/w_{\text{clip}} \end{pmatrix} = \begin{pmatrix} \frac{\frac{5\sqrt{2}}{27}}{\frac{1}{9}} \\ \frac{\frac{1}{9}}{\frac{162-22\sqrt{2}}{135}} \\ \frac{162-22\sqrt{2}}{135} \end{pmatrix} = \begin{pmatrix} 0.2619 \\ 0.1111 \\ 0.9695 \end{pmatrix}$$

视口变换 (Viewport Transformation)

- 计算点在屏幕窗口上的位置
- 参数
 - 屏幕窗口宽度width, 高度height
- 通常假设屏幕左下角的像素坐标为 (0, 0), 且变换后z坐标范围为[0, 1]
- 则

$$\begin{aligned}x_{window} &= \frac{width}{2}(x_{NDC} + 1) + x \\y_{window} &= \frac{height}{2}(y_{NDC} + 1) + y \\z_{window} &= \frac{1}{2}z_{NDC} + \frac{1}{2}\end{aligned}$$
$$v_{NDC} = \begin{pmatrix} x_{clip} / w_{clip} \\ y_{clip} / w_{clip} \\ z_{clip} / w_{clip} \\ 1 \end{pmatrix} \longrightarrow v_{window} = \begin{pmatrix} x_{window} \\ y_{window} \\ z_{window} \\ 1 \end{pmatrix}$$

vertex in NDC vertex in window coords

- 视口变换也可以使用矩阵来表示

视口变换 – 样例

- NDC坐标为 $p_{\text{ndc}} = \begin{pmatrix} x_{\text{clip}}/w_{\text{clip}} \\ y_{\text{clip}}/w_{\text{clip}} \\ z_{\text{clip}}/w_{\text{clip}} \end{pmatrix} = \begin{pmatrix} \frac{5\sqrt{2}}{27} \\ \frac{1}{9} \\ \frac{162-22\sqrt{2}}{135} \end{pmatrix} = \begin{pmatrix} 0.2619 \\ 0.1111 \\ 0.9695 \end{pmatrix}$
- 假设有分辨率200x200像素的屏幕，左下角像素坐标为(0, 0)，计算该点在屏幕空间的坐标。
- $\begin{pmatrix} x_w \\ y_w \end{pmatrix} = \begin{pmatrix} \frac{l_w}{2} x_{\text{ndc}} + \frac{l_w}{2} \\ \frac{h_w}{2} y_{\text{ndc}} + \frac{h_w}{2} \end{pmatrix} = \begin{pmatrix} 100x_{\text{ndc}} + 100 \\ 100y_{\text{ndc}} + 100 \end{pmatrix} \approx \begin{pmatrix} 126.189 \\ 111.111 \end{pmatrix}$

渲染过程中各个变换的顺序是

- ☐ A 模型变换、视口变换、投影变换、视角变换
- ☒ B 模型变换、视角变换、投影变换、视口变换
- ☐ C 视角变换、模型变换、视口变换、投影变换
- ☐ D 视角变换、模型变换、投影变换、视口变换

提交

目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

04 着色模型

05 渲染管线

06 课程小作业和大作业

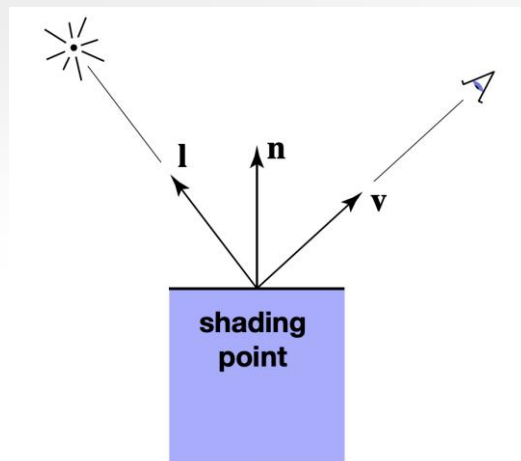
着色

- 着色 (shading)：依据物体表面与光线的夹角、物体与光源的距离、物体表面与视线的夹角、物体的材质等因素决定物体表面颜色的过程



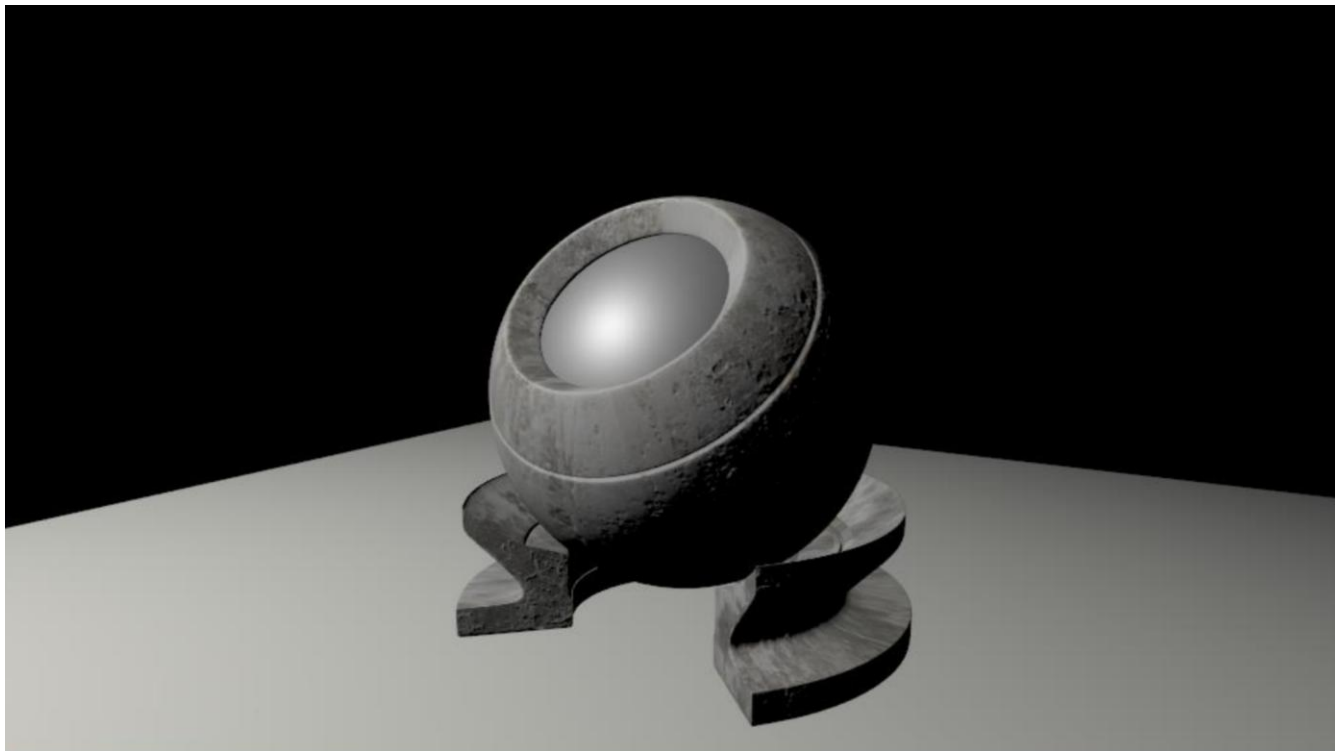
着色

- 输入
 - 视角 v
 - 表面法向量 n
 - 光线方向 l (可能有多个光源)
 - 表面属性 (颜色、光泽度、粗糙度 ...)



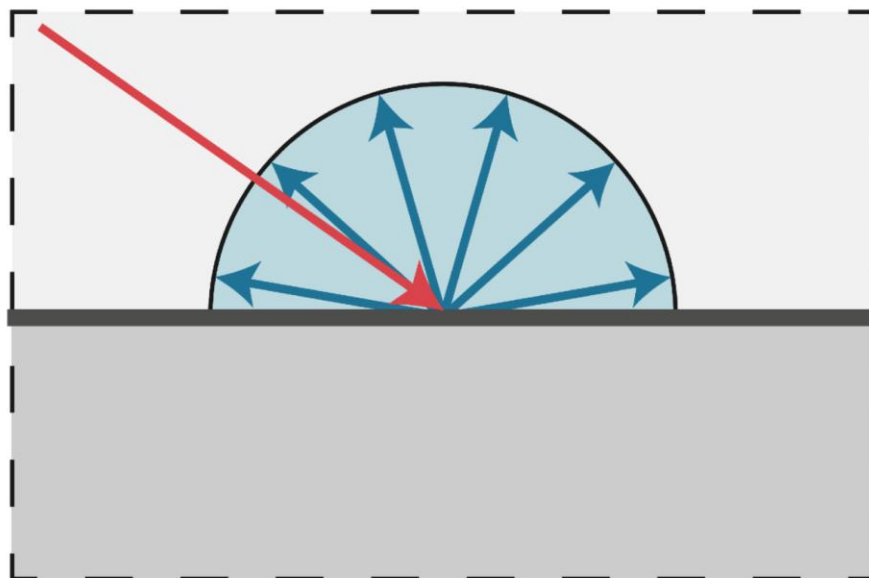
着色

- shading不是shadow



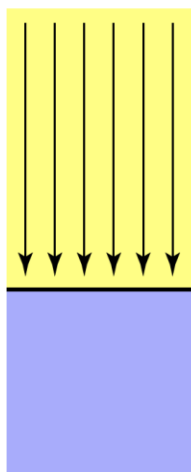
Lambertian着色模型

- 漫反射：
 - 入射光线被均匀反射至所有方向
 - 表面颜色在所有观察方向相同

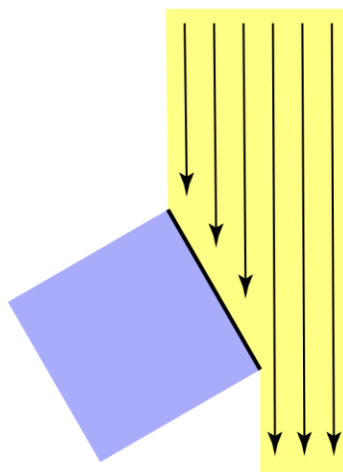


Lambertian着色模型

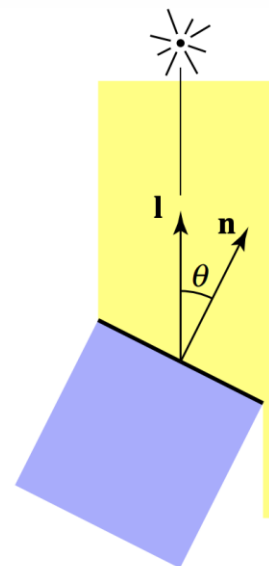
- 表面接收到的能量符合Lambert余弦定则



Top face of cube
receives a certain
amount of light



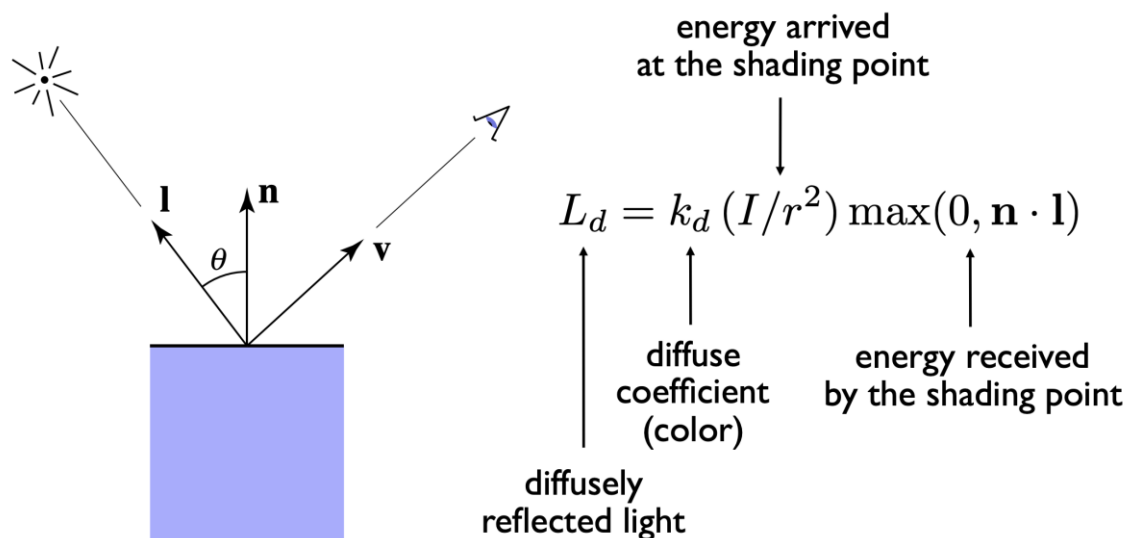
Top face of
60° rotated cube
intercepts half the light



In general, light per unit
area is proportional to
 $\cos \theta = \mathbf{l} \cdot \mathbf{n}$

Lambertian着色模型

- Lambertian着色模型，或称漫反射着色模型
- 着色与表面和光线的夹角有关，与观察方向无关

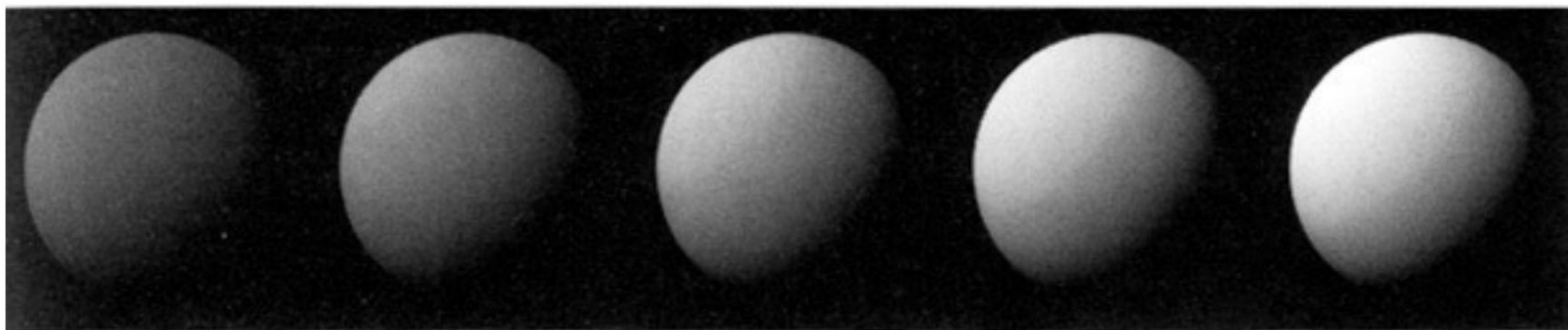


I : 距离光源单位距离的光源强度 (平方衰减)

r : 光源到该点的距离

Lambertian着色模型

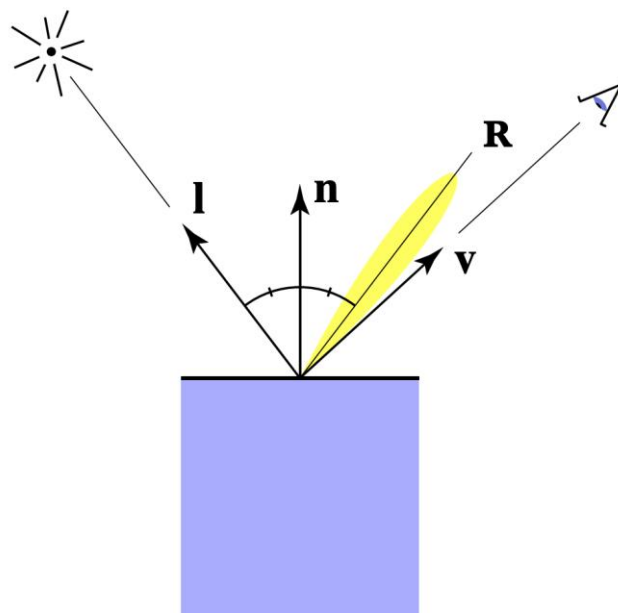
- 漫反射系数 K_d



$k_d \longrightarrow$

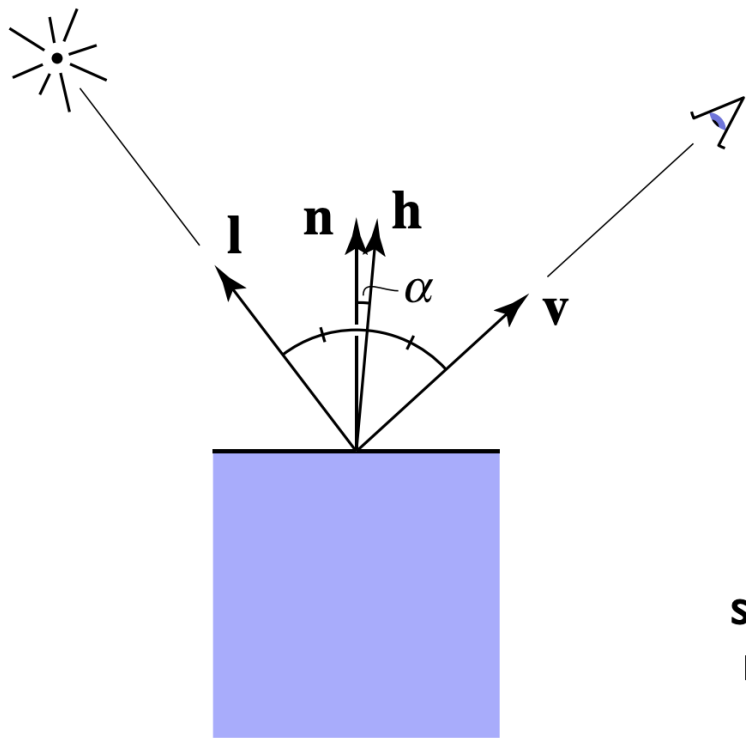
Blinn-Phong着色模型

- 相比Lambertian着色模型多了Specular高光项
- 着色与观察方向相关
- 越靠近镜面反射方向越亮



Blinn-Phong着色模型

- 视线 $\mathbf{v}$ 与镜面反射方向相近 $\rightarrow$ 半程向量与法线方向相近
- 按向量的点积衡量相近程度

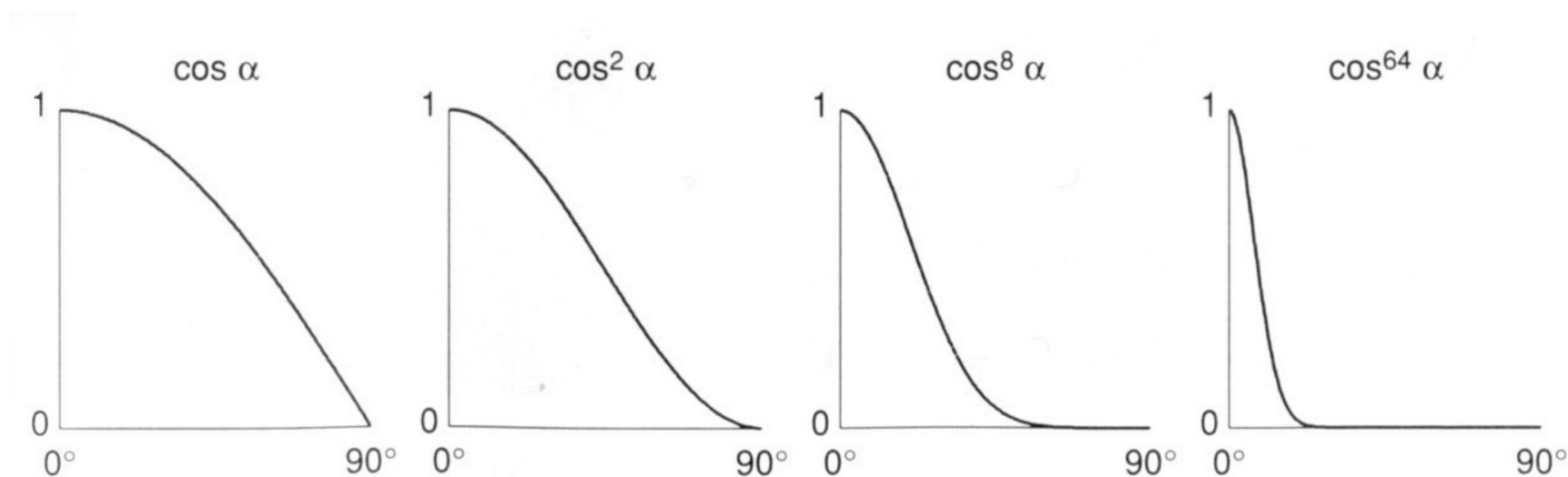


$$\begin{aligned}\mathbf{h} &= \text{bisector}(\mathbf{v}, \mathbf{l}) \\ &\text{(半程向量)} \\ &= \frac{\mathbf{v} + \mathbf{l}}{\|\mathbf{v} + \mathbf{l}\|}\end{aligned}$$

$$\begin{aligned}L_s &= k_s (I/r^2) \max(0, \cos \alpha)^p \\ &\quad \uparrow \\ &\text{specularly} \\ &\text{reflected} \\ &\text{light} \\ &= k_s (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{h})^p \\ &\quad \uparrow \\ &\text{specular} \\ &\text{coefficient}\end{aligned}$$

Blinn-Phong着色模型

- p 值越小，高光越明显

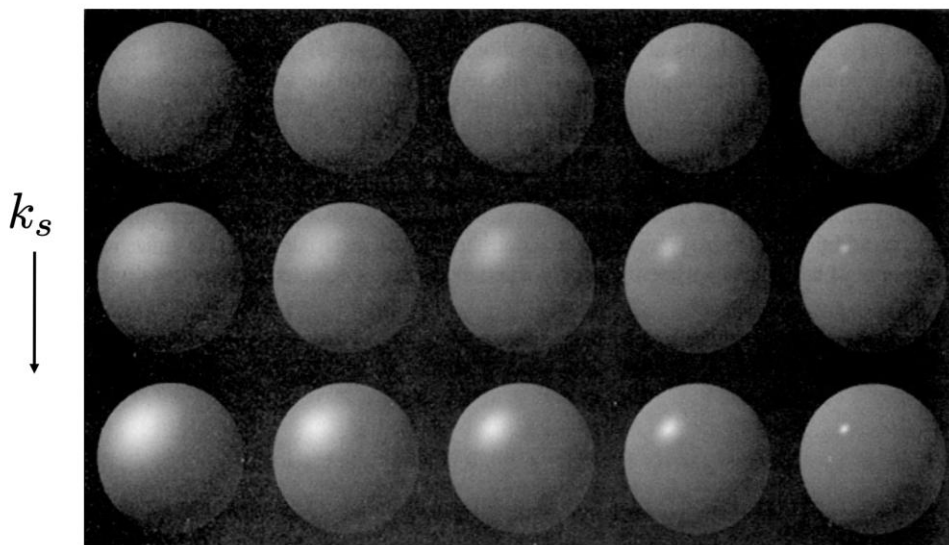


Blinn-Phong着色模型

- 不同 k_s 与 p 值组合的高光效果

Blinn-Phong

$$L_s = k_s (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{h})^p$$

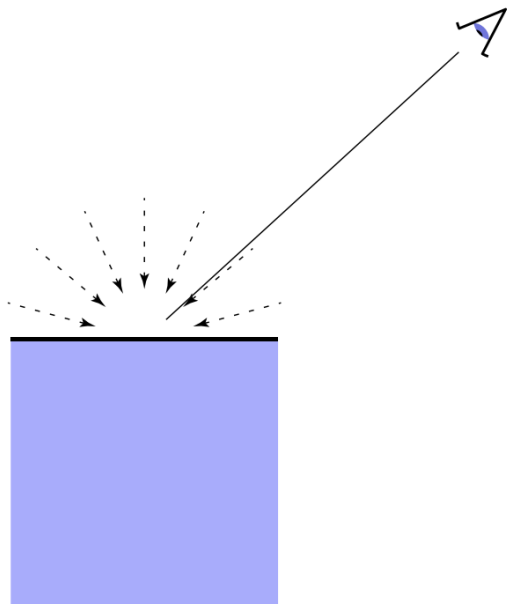


Note: showing
Ld + Ls together

$p \longrightarrow$

Blinn-Phong着色模型

- Ambient环境光项
- 不取决于任何因素的恒定着色
- 是对生活中环境光照的近似



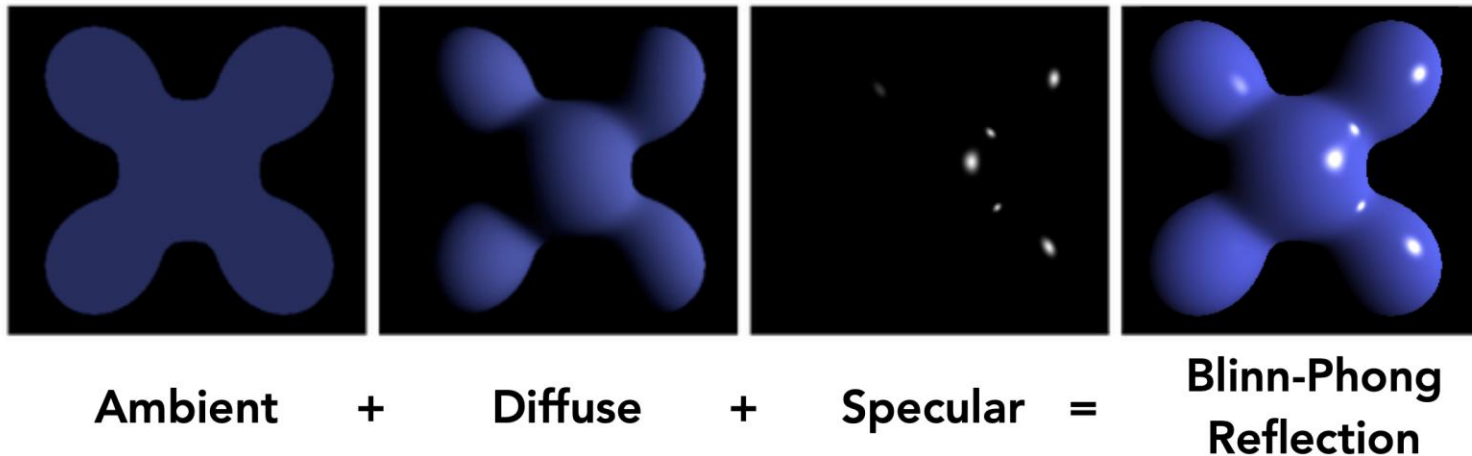
$$L_a = k_a I_a$$

↑ ↑

reflected ambient

ambient light coefficient

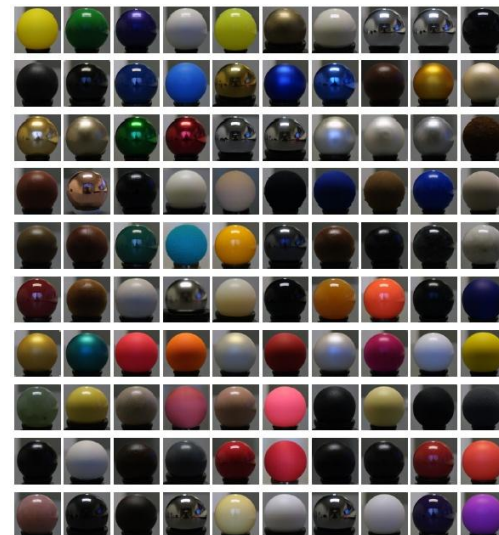
Blinn-Phong着色模型



$$\begin{aligned} L &= L_a + L_d + L_s \\ &= k_a I_a + k_d (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{l}) + k_s (I/r^2) \max(0, \mathbf{n} \cdot \mathbf{h})^p \end{aligned}$$

材质

- Blinn-Phong着色模型是对真实世界材质的一种简单近似
- 任意复杂的材质都可以用双向反射分布函数 (BRDF, Bidirectional Reflectance Distribution Function) 来表示
- BRDF $f_r(\omega_i \rightarrow \omega_r)$ 定义为任意角度 ω_r 出射光辐射度与任意角度 ω_i 有效入射光照度的比值, 通常用测量的方式获得
- 漫反射物体的BRDF是常数



Blinn-Phong模型由哪几部分组成

- ☒ A 环境光照、漫反射和高光
- ☐ B 环境光照、漫反射、镜面反射和折射
- ☐ C 漫反射、高光和凹凸贴图
- ☐ D 漫反射和高光

提交

目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

04 着色模型

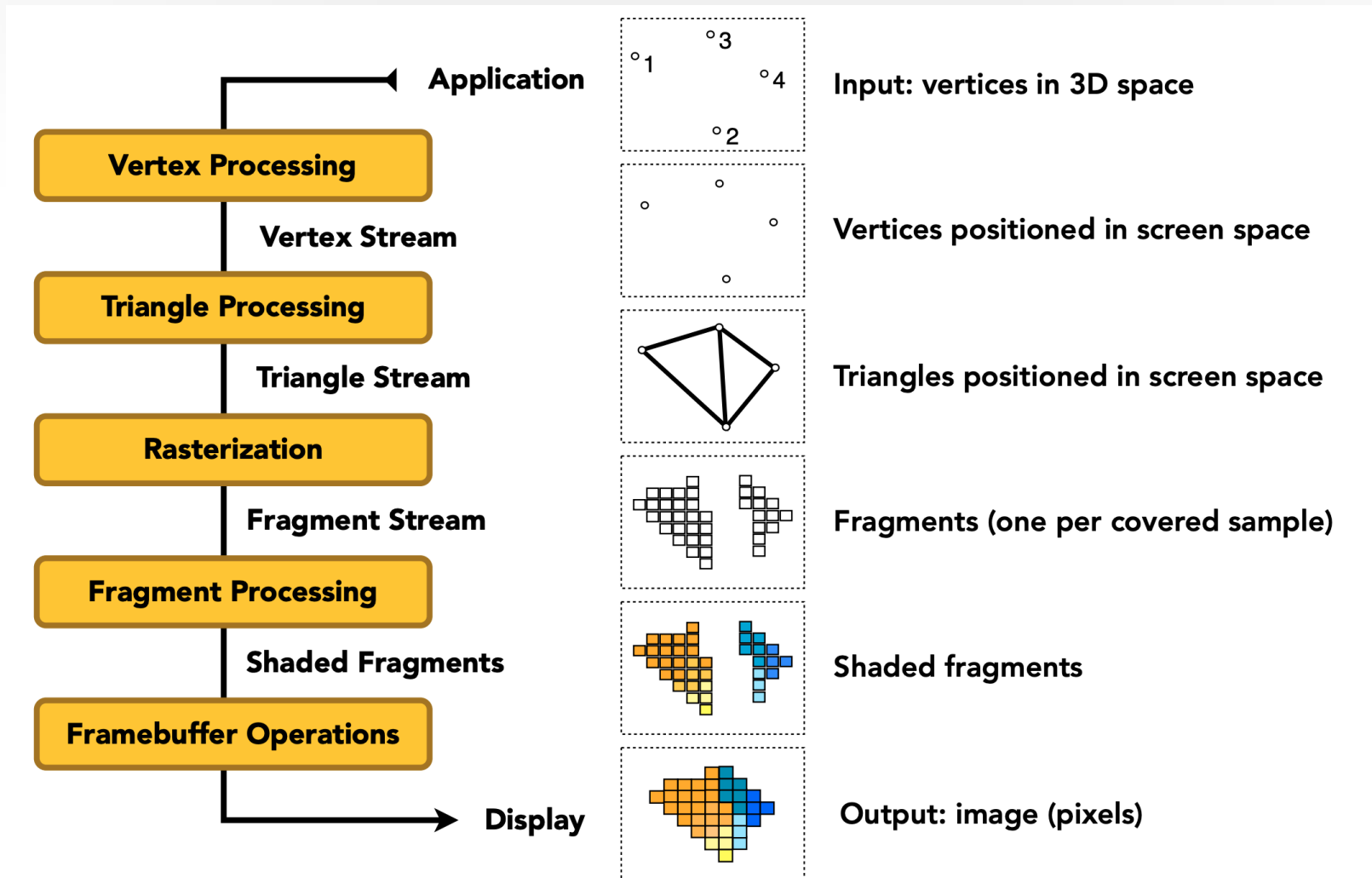
05 渲染管线

06 课程小作业和大作业

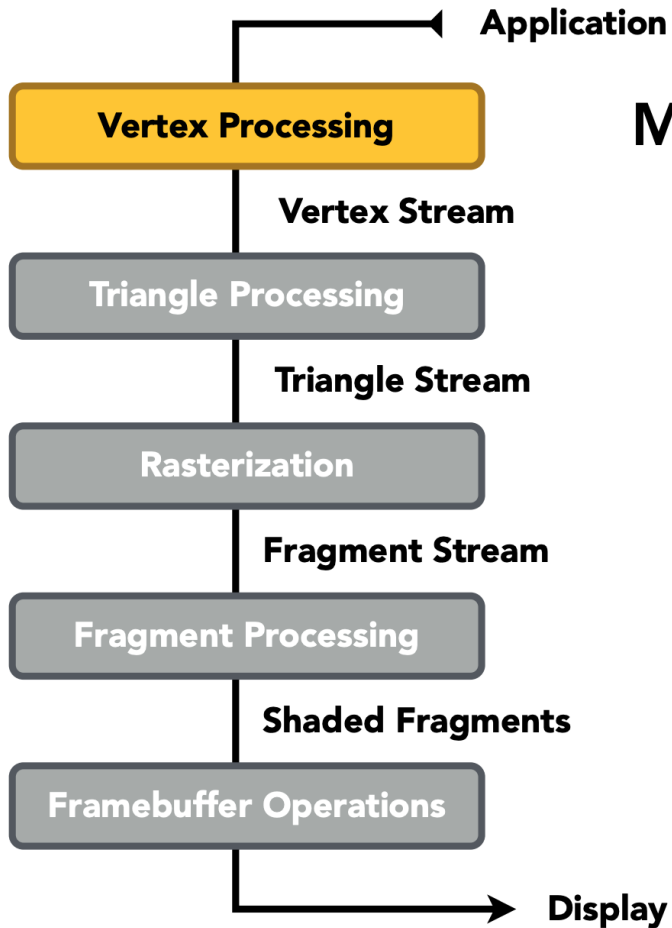
渲染管线

- 在计算机图形学中，渲染管线（rendering pipeline）是描述将3D场景渲染成2D图像所需步骤的概念模型。
- 这些步骤既依赖软件，也依赖硬件，因此没有通用的实现方案。
- Direct3D和OpenGL等图形应用编程接口（API）是针对某些特定硬件设备的实现。
- 因其有针对硬件的支持和优化，可用于实时渲染。

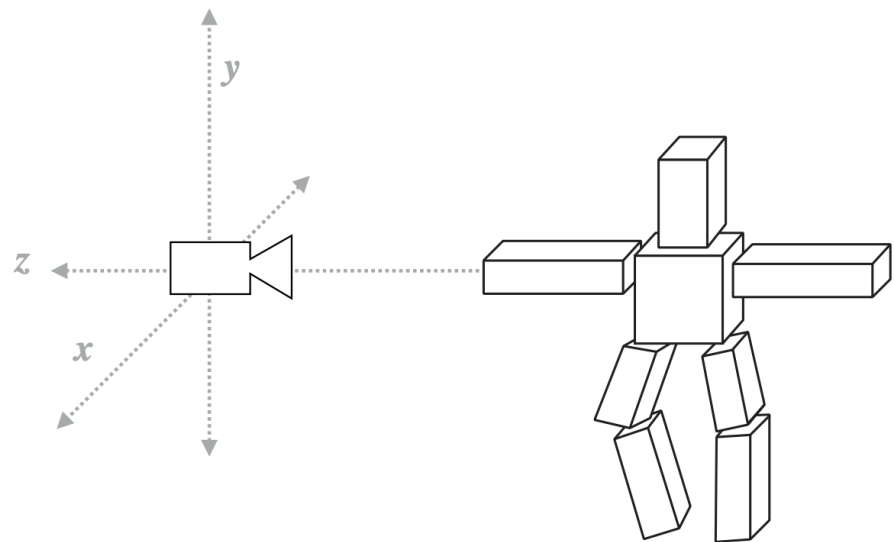
渲染管线



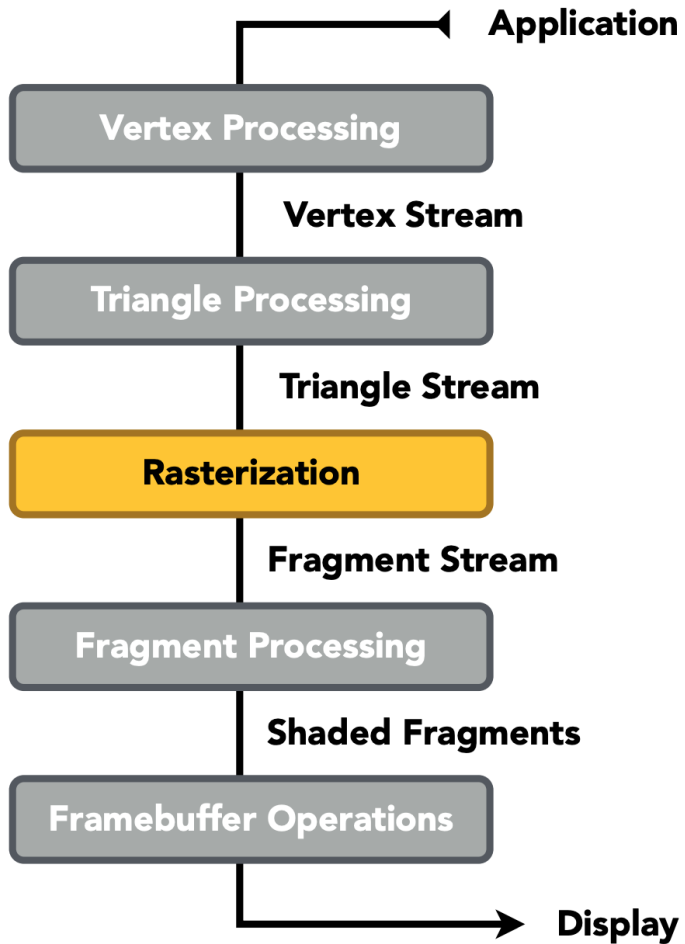
渲染管线



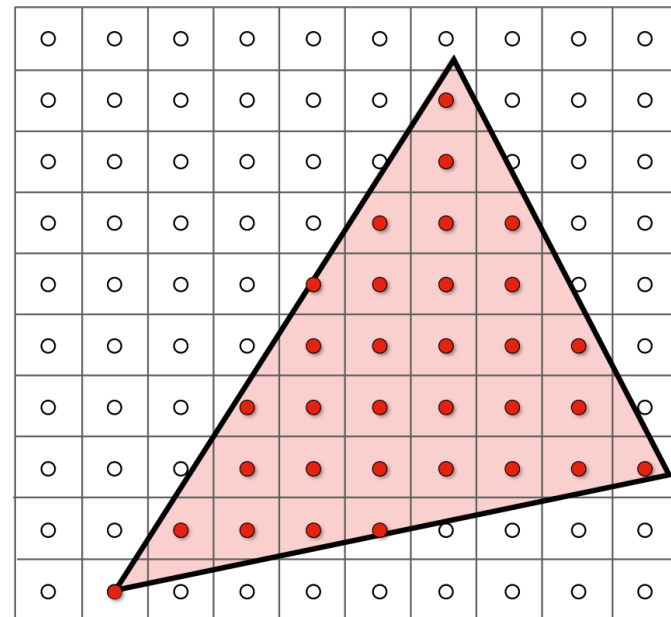
Model, View, Projection transforms



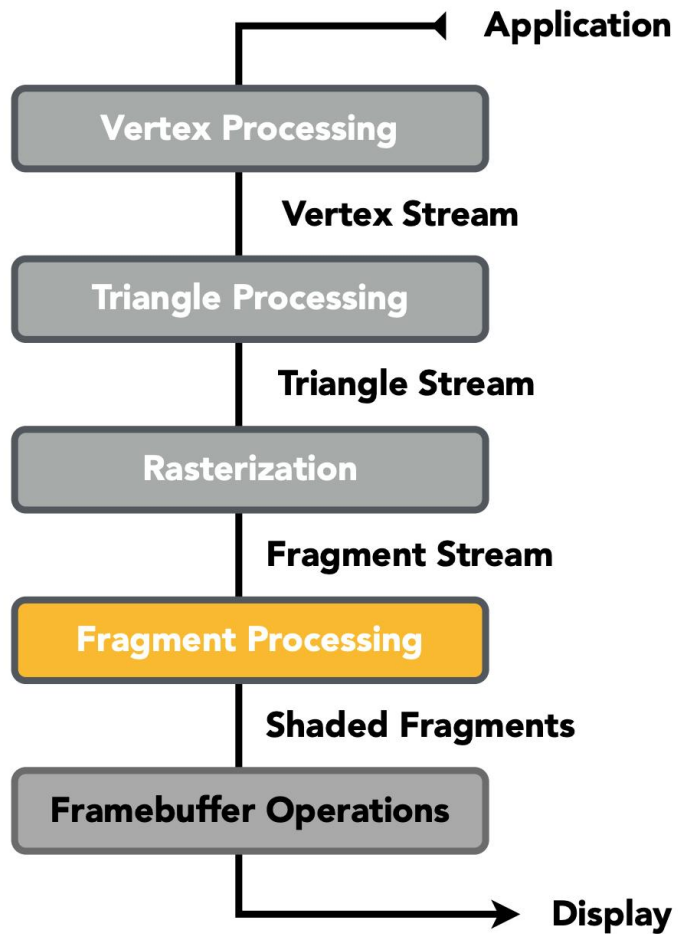
渲染管线



Sampling triangle coverage



渲染管线

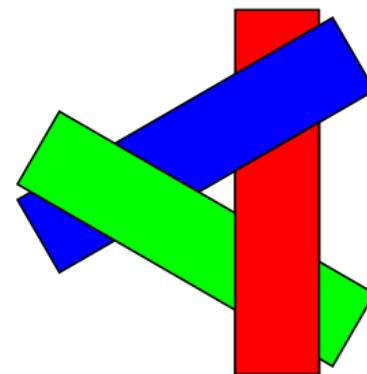
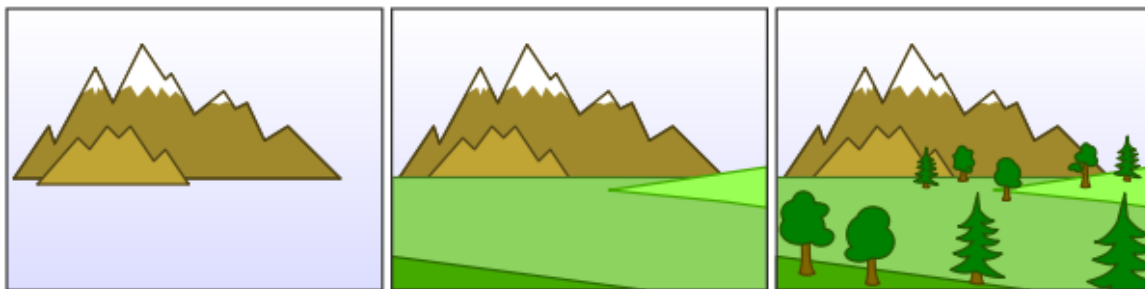


Z-Buffer Visibility Tests



渲染管线 – 可见性问题

- 如何在渲染过程中解决三角形前后顺序的问题？
 - 靠前的三角形应当遮住靠后的三角形
- 一个简单的方法：画家算法（painter's algorithm）。将三角形按照深度排序，从深到浅依次绘制。
- 问题：
 - 需要对所有三角形排序、绘制所有三角形，效率低
 - 无法解决三角形深度顺序有歧义的情况



渲染管线 - 可见性问题

- Z-Buffer算法

- 基本思想：用一个额外的缓存区（depth buffer）保存每个像素当前最小的深度值
- 伪代码：

将depth buffer的值初始化为 ∞

for (each triangle T)

for (each sample (x,y,z) in T)

```
if (z < zbuffer[x,y]) // closest sample so far
```

```
framebuffer[x,y] = rgb;           // update color
```

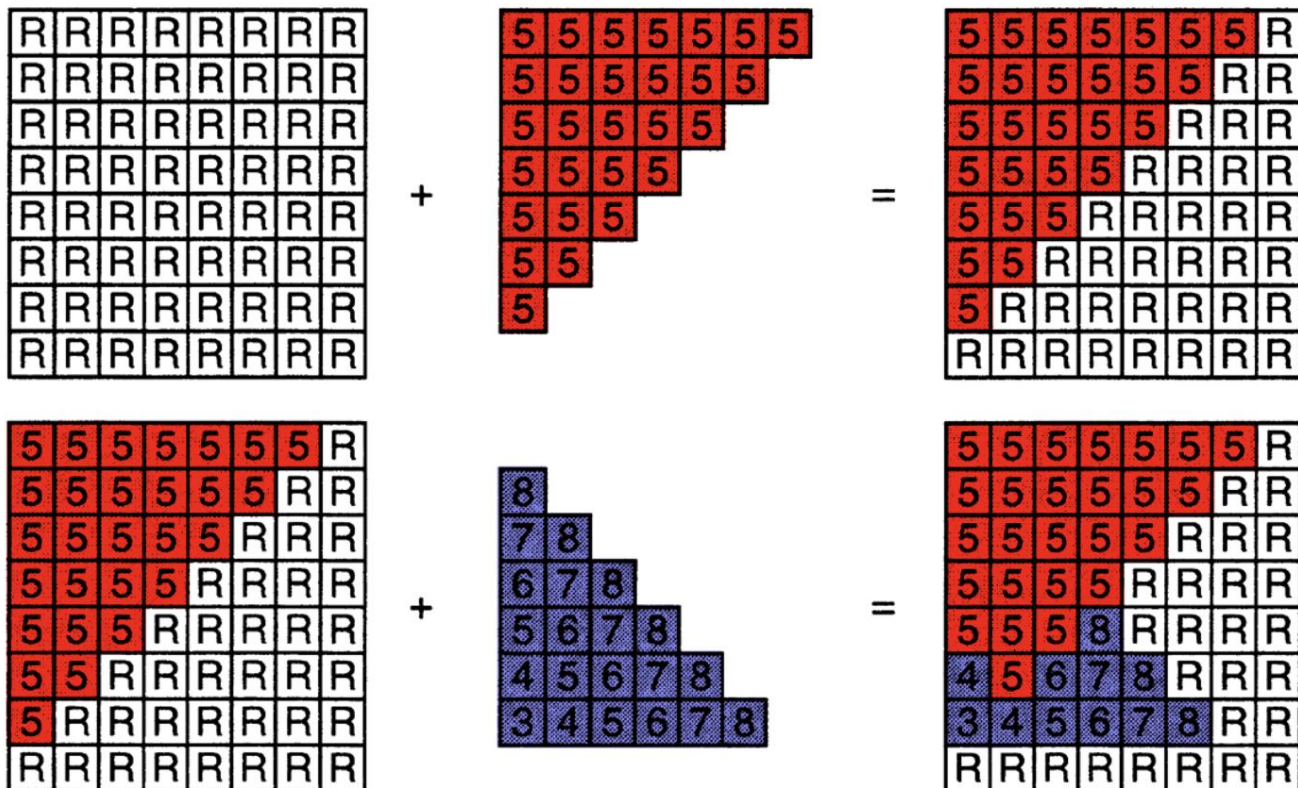
```
zbuffer[x,y] = z;           // update depth
```

else

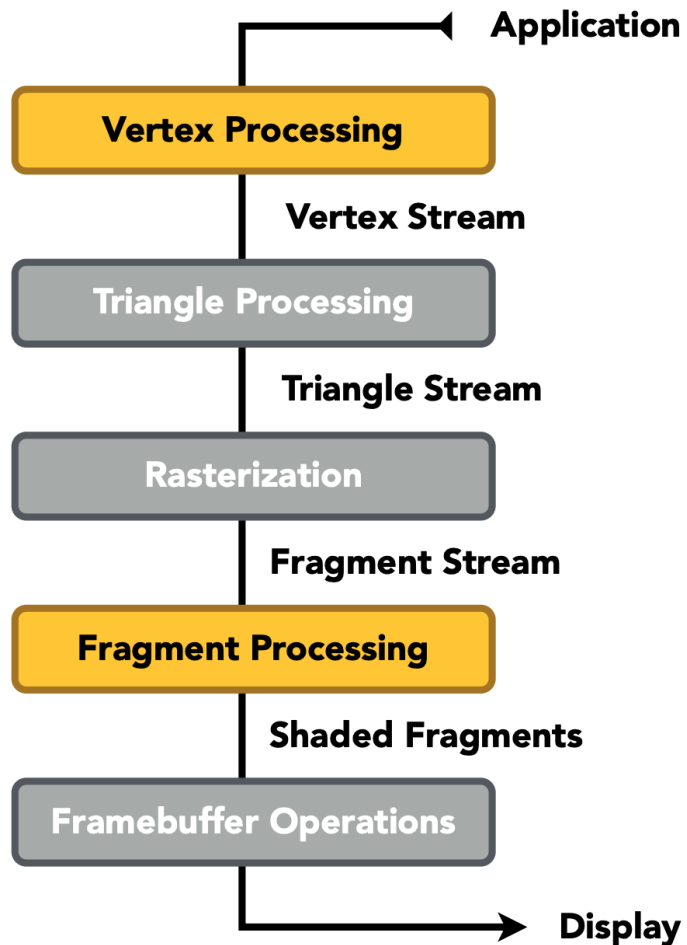
```
; // do nothing, this sample is occluded
```

渲染管线 – 可见性问题

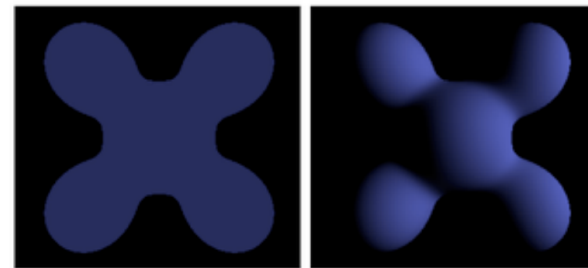
- Z-Buffer算法



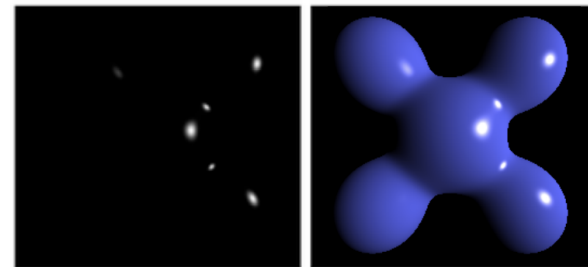
渲染管线



Shading

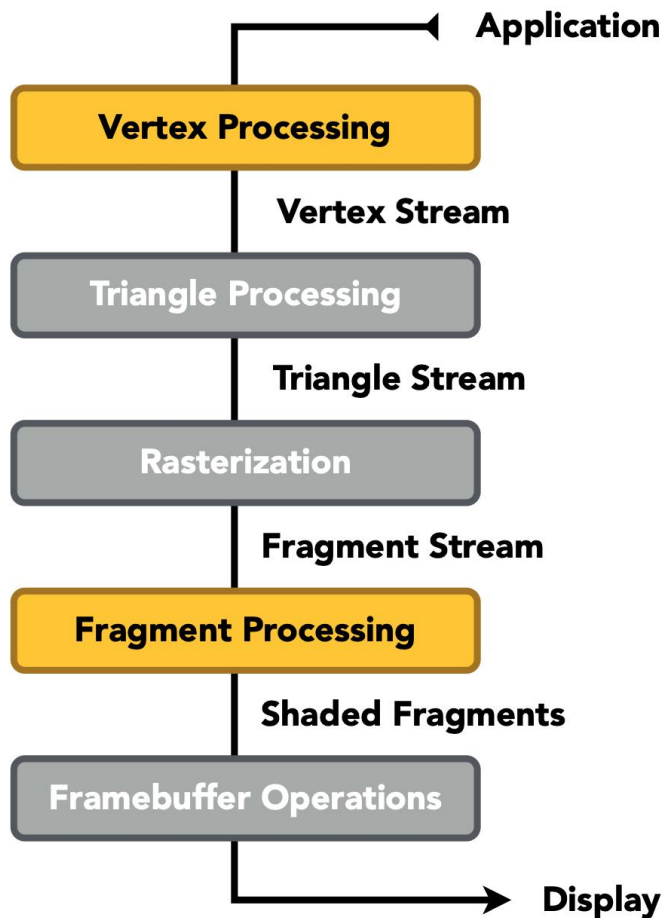


Ambient + Diffuse

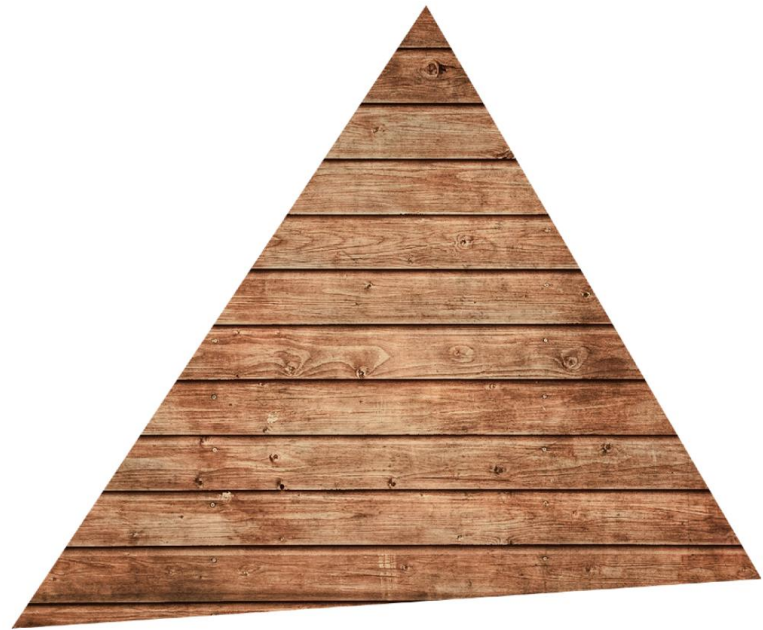


+ Specular = Blinn-Phong Reflectance Model

渲染管线



Texture Mapping



目录

01 三维模型几何表示

02 纹理

03 齐次坐标与变换

04 着色模型

05 渲染管线

06 课程小作业和大作业

课程小作业

- 目的：通过概念考察和简单编程巩固基础知识
- 编程语言
 - Python
 - C# (Unity)
- 硬件平台
 - Oculus Quest2 (3人一组, 每组一套)

本学期总体安排

序号	时间	篇章	课程内容	作业安排
1	9.15	绪论	虚拟现实技术概述	
2	9.22		课程预备知识与实验平台介绍	小作业1
3	9.29	硬件交互	视觉的机制与显示装置(I)	
4	10.13		视觉的机制与显示装置(II)	
5	10.20		听觉、触力觉机制与装置	小作业2
6	10.27		输入技术与装置、典型VR/AR系统	
7	11.3		三维空间交互技术(I)	小作业3
8	11.10		三维空间交互技术(II)	大作业
9	11.17	建模技术：基础与几何建模		
10	11.24	建模技术：物理与行为建模		
11	12.1	VR场景渲染技术		
12	12.8	全景内容的合成与应用		
13	12.15	三维场景内容合成与创作		
14	12.22	内容处理	虚拟现实的心理因素和社会伦理思考	
15	12.29	大作业展示与交流		

课程小作业 (35分)

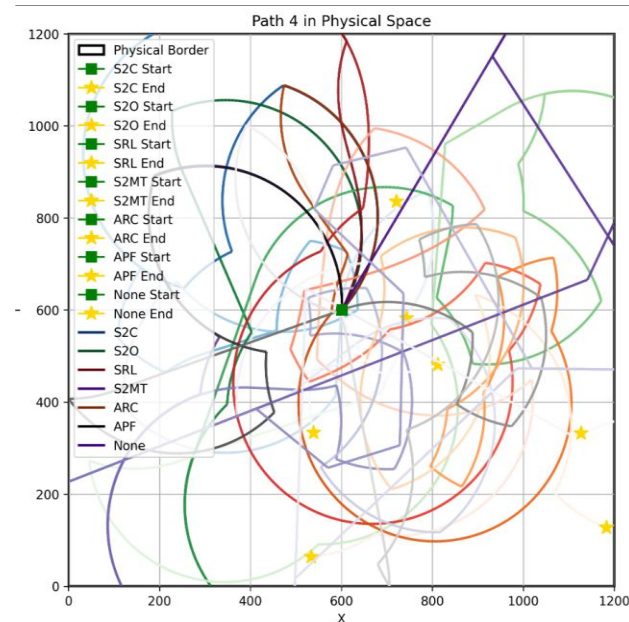
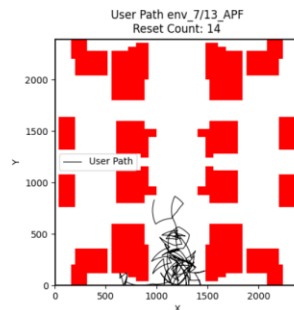
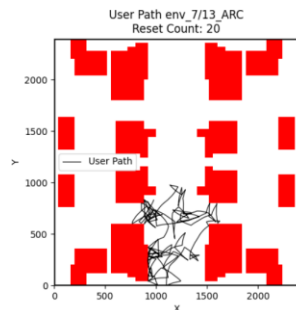
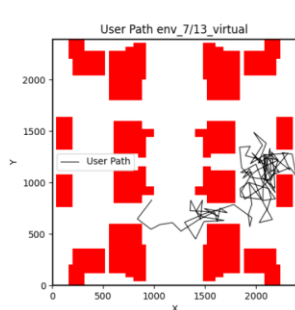
- 小作业1：绘制管线中的顶点坐标变换 (10分, 每人完成)
 - 书面作业：顶点坐标变换的计算
 - 编程作业 (Python)：顶点坐标变换代码补全
- 小作业2：立体视觉与听觉 (10分, 每人完成)
 - 书面作业：立体视觉相关概念问答
 - 编程作业 (Python)：HRTF空间立体声算法代码补全
- 小作业3：三维空间交互 (15分, 每组完成)
 - 书面作业：三维空间交互相关概念问答
 - 编程作业 (C#)：实现一个简单的三维空间交互方法

课程大作业 – 应用与探索 (60分)

- 3人一组
- 实验题目选择
 - 与虚拟现实相关的应用或科研课题探索, 不鼓励做游戏
 - 多个备选题目, 也可以自主提出选题
- 开发环境与设备
 - 每组一套Oculus Quest2, 使用Unity (C#) 进行开发
 - 也可以不使用设备

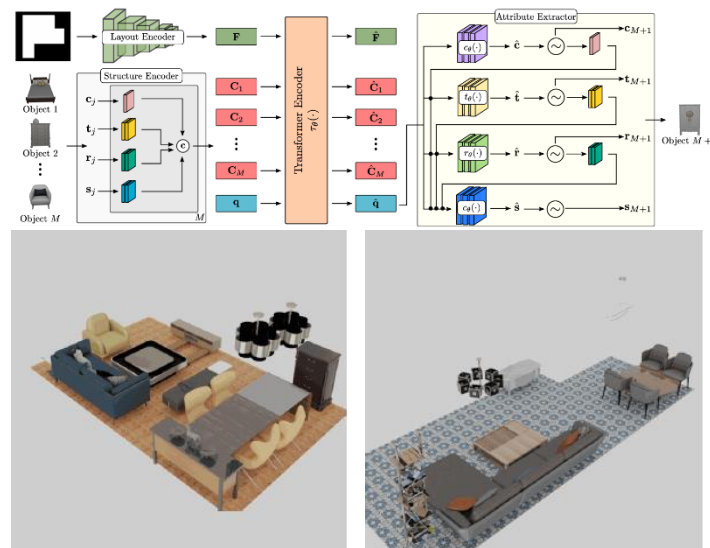
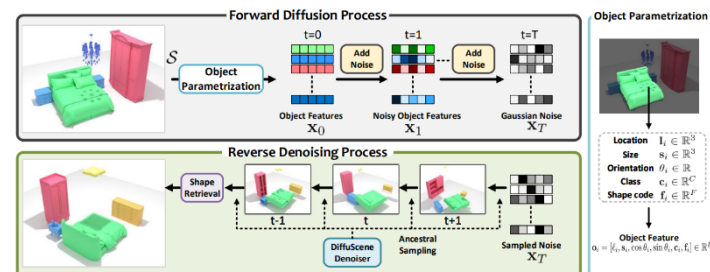
大作业预设选题——重定向行走仿真

- 基础任务（60%）：复现S2C、S2O、APF
- 进阶任务（20%）：复现较先进方法，或自研新方法
- 探索问题（20%）
 - 鼓励自研新方法（加分项）
 - 现有方法基准测试
 - 自适应控制器选择
 - 多人同空间/异空间的协同



大作业预设选题——深度学习室内三维场景生成

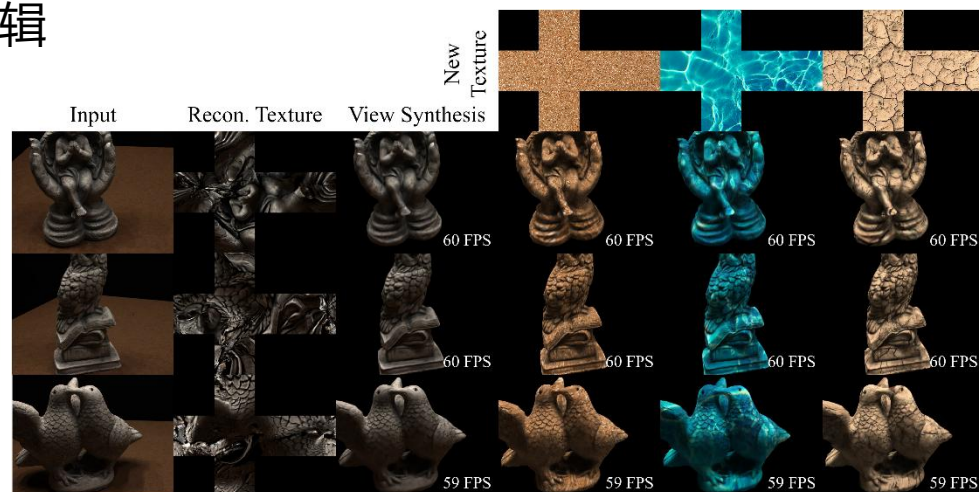
- 基础任务（60%）：加载预训练模型进行场景生成
- 进阶任务（20%）：可视化生成过程、实现条件控制生成、评估模型能力
- 探索问题（20%）
 - 自行训练模型
 - 探索条件控制生成、场景评估方法
 - 提出改进idea并进行测试、对比：
 - 模型结构
 - 训练方法
 - 数据处理
 - ...



大作业预设选题——3DGS

- 基础任务（60%）：复现已有的3D Gaussian Splatting 代码，在自己拍摄的室内图像集合上重建场景，支持在电脑上的渲染与漫游
- 进阶任务（20%）：在Oculus上支持3D Gaussian Splatting的渲染与沉浸式漫游，需要实现着色器部分的代码修改
- 探索问题（20%）
 - 重光照：在VR环境中，对于物体级别的3D Gaussian表示，支持通过修改环境光贴图来进行重光照
 - 纹理编辑：在VR环境中，对于物体级别的3D Gaussian表示，支持通过修改纹理贴图来进行纹理编辑

Relighting Results



大作业预设选题——全景图

- 基础任务（60%）：复现SIFT算法，在自己拍摄的室内图像集合上进行特征匹配，合成全景图像，在Oculus上支持观看全景图
- 进阶任务（20%）：复现柏松融合算法，能支持修复伪影和全景图像中的物体替换。
- 探索任务（20%）：运行任意一个全景图生成模型，从双目图像/单张图像生成全景图，并能够将全景图在VR设备中观看，论文参考：
 - MatryODShka: Real-time 6DoF Video View Synthesis using Multi-Sphere Images (ECCV 2020)
 - DreamCube: 3D Panorama Generation via Multi-plane Synchronization (ICCV 2025)

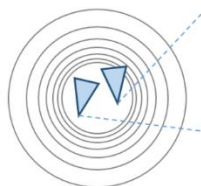
Omnidirectional Stereo (ODS)



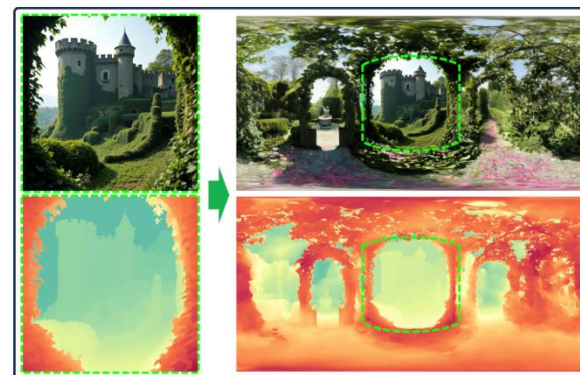
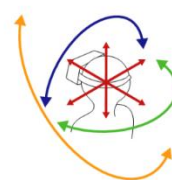
360° ODS Video



Multi-Sphere Image (MSI)



6DoF 360° View Synthesis



RGB-D Panorama Generation from Single View

大作业自由选题——VR创新式交互

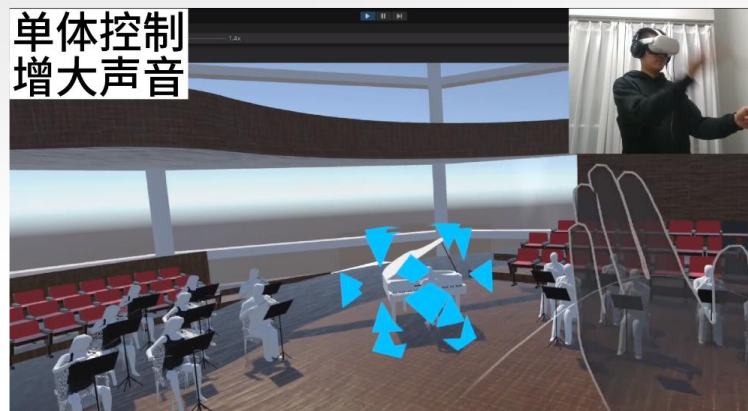
- 基础任务（60%）
 - 明确主题，在现有三维引擎中实现，并能在VR中展示
 - 相较传统交互有创新性（场景新/交互更自然或精确/沉浸感强等）
- 进阶任务（40%，以下为参考方向）
 - 创新性和趣味性
 - 多模态交互融合
 - 交互的深度与完备性
 - 用户测试与评估
 - 前沿技术创新性与挑战性

往年案例

全局控制
减小声音



单体控制
增大声音



乐团模拟器（多种手势交互+立体声）



飞行模拟器（双手柄控制，飞行逻辑实现）

大作业自由选题——VR前庭康复训练关卡设计

- 基础任务（60%）
 - 明确定义关卡要实现的前庭刺激目标，在现有三维引擎中实现，并能在VR中展示。
 - 关卡应当根据患者的实时训练情况动态调节难度，确保训练始终保持在适度的挑战水平。
- 进阶任务（40%，以下为参考方向）
 - 创新性训练机制
 - 用户体验优化
 - 数据记录与分析

参考案例



抓萝卜（手眼协同）



躲气球（侧向旋转能力）

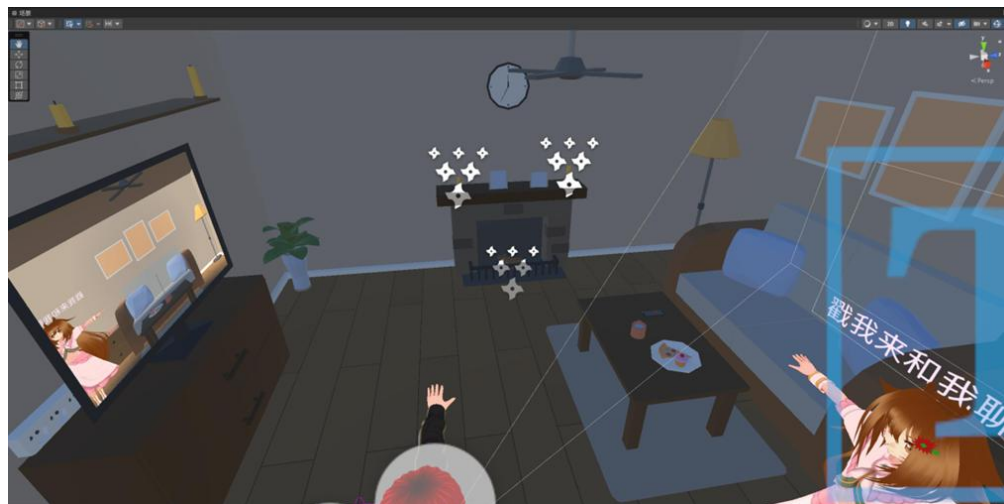
大作业自由选题——虚拟人交互

- **基础部分 (60%)**: 成功构建核心系统框架。集成大语言模型 (LLM) 作为对话引擎, 实现语音识别 (STT) 与语音合成 (TTS) 功能, 完成语音交互闭环。同时, 搭建基本VR场景并导入虚拟人模型, 最终实现用户可通过语音与虚拟人进行流畅、准确的基础问答交互。
- **进阶部分 (20%)**: 在基础交互之上, 显著提升虚拟人的表现力与真实感。实现与语音高度匹配的口型同步, 并能根据对话内容驱动虚拟人生成相应的面部表情。补充聆听、思考、说话等基础肢体动画, 并优化TTS输出, 使其语音语调富有情感变化, 超越机械播报感。
- **探索部分 (20%)**: 实现深层次的交互融合与智能行为驱动。
 - **智能肢体交互**: 虚拟人能根据对话上下文自主产生符合语义的肢体动作, 例如在介绍特定物体时, 能准确用手指向场景中的对应物体。
 - **多模态指令理解**: 虚拟人能综合理解用户的**语音指令**和**肢体动作信息** (如用户手指方向), 并做出统一回应。
 - **语音任务驱动**: 用户可通过纯语音命令指挥虚拟人操作或改变场景中的元素 (如 “打开那盏灯”、 “请把那个模型放大”), 虚拟人需理解指令并执行相应的复杂逻辑操作。

往年案例



AI模型接入、反向动力学骨骼映射、风格化渲染、衣物/头发物理解算



情感计算与AI结合、虚拟人动画交互、场景交互、VR骨骼绑定



THANKS!

清华大学计算机系

张松海

shz@Tsinghua.edu.cn